



國立臺北科技大學

資訊工程系碩士班

碩士學位論文

**基於案例式學習與動態計畫修復之 API 任
務大型語言模型代理人自我修正機制研究
A Self-Correction Mechanism for LLM Agents in
API Tasks Using Case-Based Learning and Dynamic
Plan Repair**

研究生：陳安琪

指導教授：劉建宏博士

中華民國一百一十五年七月



國立臺北科技大學

資訊工程系碩士班

碩士學位論文

**基於案例式學習與動態計畫修復之 API 任
務大型語言模型代理人自我修正機制研究**
**A Self-Correction Mechanism for LLM Agents in
API Tasks Using Case-Based Learning and Dynamic
Plan Repair**

研究生：陳安琪

指導教授：劉建宏博士

中華民國一百一十五年七月

「學位論文口試委員會審定書」掃描檔

審定書填寫方式以系所規定為準，但檢附在電子論文內的掃描檔須具備以下條件：

1. 含指導教授、口試委員及系所主管的完整簽名。
2. 口試委員人數正確，碩士口試委員至少 3 人、博士口試委員至少 5 人。
3. 若此頁有論文題目，題目應和書背、封面、書名頁、摘要頁的題目相符。
4. 此頁有無浮水印皆可。

摘要

關鍵詞：大型語言模型代理人、API 任務自動化、案例式學習、動態修復

大型語言模型 (Large Language Model, LLM) 具備自然語言理解、推理與內容生成能力；當其作為代理人 (agent) 的決策核心時，可根據使用者目標進行任務規劃與工具選擇，並透過應用程式介面 (Application Programming Interface, API) 與外部系統互動。然而，在多步驟任務中，任一步驟的 API 選擇、參數綁定、資料處理或相依關係發生錯誤，都可能使後續執行偏離原始目標，因此如何利用既有成功案例輔助規劃，並針對局部失敗進行動態修復，是提升 API 任務代理人可靠度的重要議題。本研究以 API 知識圖譜代理人 (API Knowledge Graph Agent, APIKGAgent) 為基礎，提出案例式修復 API 知識圖譜代理人 (Case-Based Repair APIKGAgent, CBR-APIKGAgent)，首先從訓練資料中的成功任務萃取可跨任務重複使用的原子化經驗 (atomic experience)，並透過語意檢索提供給初始規劃、重新規劃與修復階段參考；其次設計局部修復流程，根據執行錯誤、相依步驟資料與 API 呼叫證據搜尋可能的修復 API，並以插入、取代或刪除步驟的方式調整局部計畫，當錯誤涉及原始計畫結構時則回退至重新規劃。為評估本研究方法，實驗採用 AppWorld 模擬環境，其涵蓋 9 種應用服務與 457 支 API，可呈現跨應用程式的現實多步驟任務；評估以任務是否完成並使環境產生預期狀態變化為準，並透過整體效能比較、逐步消融實驗與失敗案例分析，觀察局部修復與原子化經驗對任務執行的影響。研究結果顯示，在 AppWorld test_normal 資料集上，本研究取得完整任務成功率 X%；此外，局部修復對有明確錯誤訊號的情境有效，但面對較複雜的任務結構或外部狀態限制時，僅靠局部修復與經驗提示仍可能不足以穩定恢復。

ABSTRACT

Keyword: LLM agents, API task automation, case-based learning, dynamic plan repair

Large language models (LLMs) have recently been widely adopted in agent systems, where they can plan tasks, select tools, and invoke APIs according to user goals to complete real-world cross-application tasks. In multi-step API tasks, however, an error in API selection, parameter binding, data processing, or dependency handling at any step may cause the subsequent execution to deviate from the original objective. Traditional approaches usually regenerate the entire plan after an error, which may increase execution cost and disrupt the state and data flow accumulated from completed steps. Therefore, incorporating case-based guidance and local repair mechanisms into an existing planning framework is an important issue for improving the reliability of API task agents. Based on the API Knowledge Graph Agent (APIKGAgent), this study proposes the Case-Based Repair APIKGAgent (CBR-APIKGAgent). The proposed system first extracts reusable atomic experiences from successful tasks in the training data and provides them as references for initial planning, replanning, and repair through semantic retrieval. It then introduces a local repair process that searches for possible repair APIs according to execution errors, dependency information from related steps, and API call evidence, and adjusts the local plan through insertion, replacement, or deletion of steps; when an error affects the structure of the original plan, the system falls back to replanning. Experiments were conducted in the AppWorld simulation environment, which contains 9 application services and 457 APIs and reflects realistic multi-step cross-application tasks. Performance was evaluated based on task completion and the expected state changes in the environment, and the effects of local repair and atomic experience were examined through overall performance comparison, stepwise ablation studies, and failure-case analysis. On the AppWorld test_normal dataset we observe an overall task success rate of X%; local repair helps when errors have explicit signals but may be insufficient for failures caused by complex task structures or external state constraints.

誌謝

回顧這段碩士求學的歷程，從最初的摸索到最終完成論文，其中經歷了無數嘗試與修正。能夠走到這一步，並非一人之力，而是來自許多人的陪伴與支持。

首先，誠摯感謝指導教授劉建宏老師。在研究過程中，老師總能在關鍵時刻點出問題核心，引導我重新思考方向。無論是研究上的討論，或是面對困難時的提醒，都讓我逐漸建立起更成熟的思考方式，也學會如何更嚴謹地面對學術問題。同樣感謝梁文耀教授的指導與建議。您在討論中提出的觀點，往往讓我跳脫原有的框架，使研究內容得以更加周全，也讓我對自己的研究有更深一層的理解。

在實驗室的日子裡，感謝珮倫、郁喬、冠穎、以謙等夥伴的陪伴。無論是討論問題時的腦力激盪，或是日常相處中的交流與玩笑，都讓這段研究生活不再只是壓力與進度，而多了許多值得回味的片段。此外，也想感謝一路上關心與支持我的朋友們，在我感到疲憊或迷惘時給予鼓勵，讓我能重新調整步伐，繼續前進。

最後，將這份成果獻給我的家人。謝謝你們長久以來的理解與支持，讓我能專心投入學業，在面對挑戰時依然保有前進的力量。這段旅程的結束，同時也是另一段路的開始。謹以此文，致上我最真摯的感謝。

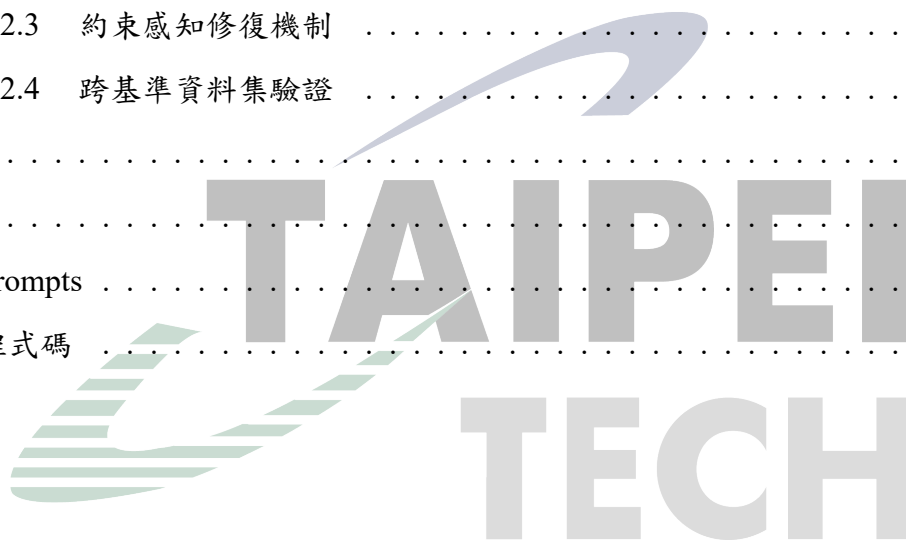
目錄

摘要	i
ABSTRACT	ii
誌謝	iii
目錄	iv
圖目錄	viii
表目錄	ix
第一章 緒論	1
1.1 研究動機	1
1.2 研究目的	2
1.3 論文組織架構	3
第二章 相關技術與文獻探討	4
2.1 大型語言模型代理人	4
2.1.1 大型語言模型代理人概述	4
2.1.2 工具使用與任務規劃	4
2.1.3 大型語言模型代理人相關研究	5
2.2 API 任務自動化	5
2.2.1 API 任務特性	6
2.2.2 API 任務自動化相關研究	6
2.2.3 AppWorld 基準環境	6
2.3 案例式學習與經驗重用	7
2.3.1 案例式推理	7
2.3.2 經驗重用相關研究	8
2.3.3 語意檢索方法	8
2.4 動態計畫修復與自我修正	9
2.4.1 重新規劃機制	9
2.4.2 自我修正機制	10
2.4.3 動態計畫修復相關研究	11

2.5	APIKGAgent	11
2.5.1	APIKGAgent 架構概述	11
2.5.2	APIKGAgent 之研究缺口	12
第三章	CBR-APIKGAgent 系統架構與方法	14
3.1	CBR-APIKGAgent 系統總覽	14
3.1.1	系統架構與核心機制	14
3.1.2	整體任務執行演算法	15
3.2	原子化經驗重用機制	18
3.2.1	經驗來源與使用範圍	18
3.2.2	原子化經驗定義	19
3.2.3	經驗檢索視角與使用情境	21
3.3	動態局部修復機制	22
3.3.1	局部修復設計目標與適用邊界	22
3.3.2	局部修復與重新規劃之選擇原則	23
3.3.3	局部計畫修復策略	23
第四章	CBR-APIKGAgent 系統實作	27
4.1	原子化經驗庫建構實作	27
4.1.1	訓練資料成功任務軌跡萃取	27
4.1.2	原子化經驗資料格式	28
4.1.3	經驗向量化與雙索引建構	28
4.1.4	經驗來源與索引使用	29
4.2	初始規劃實作	29
4.2.1	API 搜尋與候選 API 整理	29
4.2.2	規劃導向經驗檢索	30
4.2.3	初始計畫生成與提示注入	30
4.3	步驟執行與任務狀態管理	31
4.3.1	步驟執行流程	31
4.3.2	執行結果與相依資料保存	31
4.3.3	錯誤處理狀態資訊	31

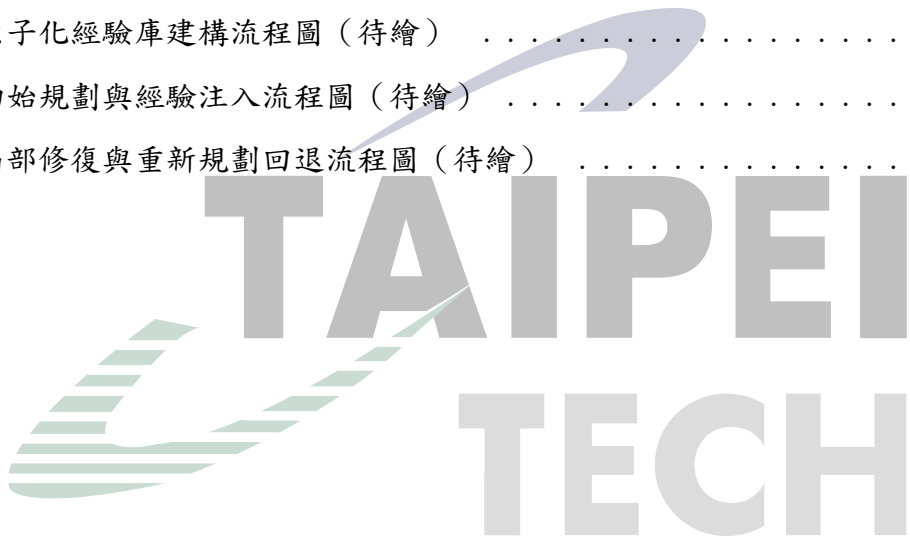
4.4	失敗處理與局部修復實作	32
4.4.1	錯誤分類與可行回饋產生	32
4.4.2	局部修復觸發與限制檢查	33
4.4.3	修復 API 搜尋與候選整理	33
4.4.4	修復導向經驗注入	33
4.4.5	插入、替換與刪除策略實作	33
4.5	重新規劃實作	35
4.5.1	重新規劃觸發條件	35
4.5.2	重新規劃提示內容	35
4.5.3	規劃導向與修復導向經驗注入	35
4.5.4	重新規劃後的流程銜接	36
第五章	實驗與結果分析	37
5.1	研究問題	37
5.2	實驗設置	38
5.2.1	資料集與任務範圍	38
5.2.2	評估指標	38
5.2.3	實驗環境與主要控制設定	39
5.3	實驗一：整體效能與失敗概況分析	40
5.4	實驗二：消融實驗與案例分析	41
5.4.1	實驗組別	42
5.4.2	消融實驗結果	42
5.4.3	原子化經驗之效益分析	43
5.4.4	動態局部修復之效益分析	43
5.4.5	兩項機制之綜合效益分析	44
5.4.6	消融實驗案例分析	44
5.5	討論	46
5.5.1	原子化經驗的影響與適用情境	46
5.5.2	動態局部修復的影響與適用情境	47
5.5.3	系統限制與能力邊界	47

第六章	結論與未來研究	48
6.1	結論	48
6.1.1	研究成果總結	48
6.1.2	研究貢獻	48
6.1.3	研究限制	48
6.2	未來研究	48
6.2.1	修復經驗累積機制	48
6.2.2	多輪局部修復策略	48
6.2.3	約束感知修復機制	48
6.2.4	跨基準資料集驗證	48
參考文獻		49
附錄		51
6.3	Prompts	51
6.4	程式碼	51



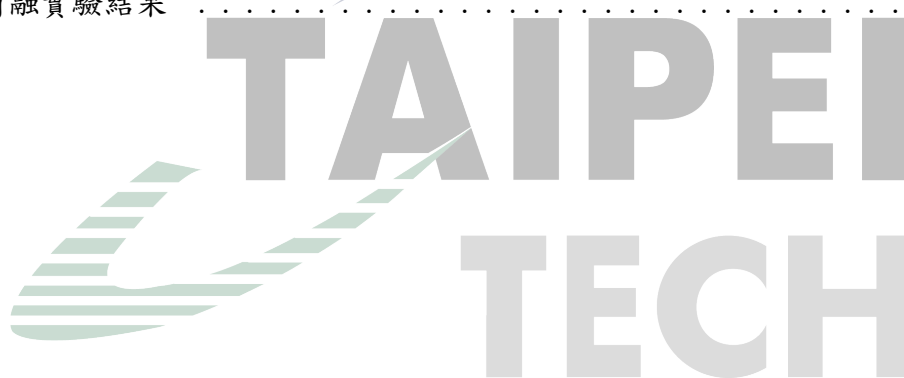
圖目錄

2.1	APIKGAgent 系統架構圖（改繪自 [2]）	12
3.1	CBR-APIKGAgent 系統架構圖	15
3.2	插入步驟之局部計畫修改示意圖	24
3.3	替換步驟之局部計畫修改示意圖	25
3.4	刪除步驟之局部計畫修改示意圖	26
4.1	原子化經驗庫建構流程圖（待繪）	29
4.2	初始規劃與經驗注入流程圖（待繪）	30
4.3	局部修復與重新規劃回退流程圖（待繪）	34



表目錄

3.1	CBR-APIKGAgent 兩項核心設計之架構角色	15
3.2	原子化經驗主要資訊面向與用途	20
4.1	錯誤處理與局部修復相關狀態資訊	32
5.1	硬體與軟體環境	39
5.2	實驗主要控制設定	39
5.3	各方法於 AppWorld test_normal 之整體效能比較	40
5.4	CBR-APIKGAgent 於 test_normal 之任務結果拆解	41
5.5	消融實驗結果	42



第一章 緒論

1.1 研究動機

近年來，大型語言模型（Large Language Model, LLM）已逐漸由傳統的文字生成與問答系統，發展為能夠自主規劃、呼叫工具並操作外部系統的大型語言模型代理人（Large Language Model Agent, LLM Agent）[1]。透過應用程式介面（Application Programming Interface, API），代理人得以執行資料查詢、訊息傳送、檔案管理、行事曆管理及交易處理等實際工作，使大型語言模型由單純提供資訊的輔助工具，進一步成為能與數位環境互動並完成使用者目標的自動化系統。因此，如何提升代理人在真實環境中的任務完成能力與執行可靠性，已成為大型語言模型研究的重要方向之一。

然而，真實世界中的 API 任務通常涉及多個相互依賴的執行步驟，而非單次 API 呼叫即可完成。代理人除了需要理解使用者需求並選擇適當的 API 外，還必須進行多步驟規劃、維護步驟間的資料依賴關係、保存中間執行結果，並持續遵循任務條件與限制。當任務規模增加或操作流程變得複雜時，任何一個步驟的錯誤都可能影響後續決策，使錯誤沿著執行流程持續傳播，最終導致整體任務失敗。因此，如何降低錯誤傳播對任務執行結果的影響，已成為提升多步驟 API 代理人可靠性的重要挑戰。

本研究延續 API 知識圖譜代理人（API Knowledge Graph Agent, APIKGAgent）[2] 之研究架構進行探討。APIKGAgent 透過 API 知識圖譜、語意檢索及任務規劃機制協助代理人完成多步驟 API 任務，並具備基本的重新規劃能力。然而，在複雜任務執行過程中仍可觀察到錯誤傳播與局部失敗難以修復等現象。此外，既有架構尚未系統化利用由訓練資料成功任務軌跡所萃取的策略經驗。此現象顯示，除了提升即時規劃能力之外，如何利用由訓練資料成功任務軌跡所萃取的經驗知識輔助決策，以及如何針對局部錯誤進行修復，亦可能是影響整體任務完成能力的重要因素。

綜合上述觀察，目前多步驟 API 任務代理人仍面臨兩項關鍵挑戰：

- **全域經驗利用不足：**本研究接續之基礎架構缺乏對訓練資料中成功任務經驗的系統化利用。代理人在不同任務中經常面臨相似的規劃與資料依賴問題，但既有系統尚未將成功任務軌跡所包含的策略知識整理為可檢索的經驗，使代理人在面對相似任務結構時，缺乏可供規劃與修復參考的依據。
- **局部錯誤修復能力不足：**當任務執行過程中發生錯誤時，現有系統多仰賴重新產生整體計畫進行修正。然而，許多錯誤並非來自整體計畫設計不良，而可能僅發生於部分步驟。若直接重新規劃整體任務，可能造成不必要的整體變動，也可能破壞原本正確的執行流程，甚至導致新的錯誤產生。

基於上述觀察，本研究認為提升多步驟 API 代理人可靠性的關鍵，不僅在於大型語言模型本身的推理能力，也在於如何利用由訓練資料成功任務軌跡所萃取的策略知識，並針對執行期間的局部錯誤進行適當修復。因此，本研究探討如何從訓練資料中

的成功任務軌跡萃取細粒度經驗，建立離線全域經驗庫，並透過語意檢索將相關經驗應用於初始規劃、重新規劃與局部修復流程；同時研究如何根據執行結果、步驟相依資料與錯誤證據進行動態局部修復。藉由經驗重用與動態局部修復機制的結合，本研究將探討全域經驗是否能為相似任務結構提供有效的規劃與修復參考，以及局部修復是否有助於代理人由執行失敗中恢復，進而評估兩項機制對多步驟 API 任務完成能力與錯誤修復表現的影響。

1.2 研究目的

本研究針對大型語言模型代理人在多步驟 API 任務中所面臨之錯誤傳播、訓練資料中成功任務策略尚未被系統化利用，以及局部失敗不易修復等問題，提出案例式修復 API 知識圖譜代理人（Case-Based Repair APIKGAgent, CBR-APIKGAgent）。其中，CBR 指 Case-Based Repair，表示本研究承接案例式學習與案例式推理中的檢索與重用概念，並將其應用於 API 任務的規劃與修復流程。本研究延續 APIKGAgent 之架構進行探討，並在其 API 知識圖譜、API 檢索、任務規劃與步驟式執行流程上，加入經驗重用與動態局部修復機制，以提升代理人在複雜 API 任務中的自我修正能力。

具體而言，本研究之目的如下：

1. 建立訓練資料成功經驗的萃取與重用機制

本研究從訓練資料中的成功任務軌跡萃取可跨任務重用的原子化經驗（Atomic Experience），將適用情境、規劃原則、必要中間概念與常見陷阱轉換為結構化經驗知識。透過語意檢索機制，系統可在初始規劃、重新規劃與局部修復階段取得與當前情境相關的經驗，以提供代理人規劃與修復時的策略參考，並探討此機制是否有助於減少已知失敗模式在相似任務中出現。

2. 建立動態局部修復機制以處理執行失敗

本研究探討代理人在執行失敗時，如何根據失敗步驟、錯誤訊息、相依資料與 API 執行結果評估可行的修復方向。當系統能形成局部調整方案時，透過插入、替換或刪除步驟修正局部計畫；當局部修復無法形成可行修正、修復次數達到限制，或錯誤訊號不足以支持局部調整時，則回退至重新規劃，以減少不必要的整體重新規劃，並降低重新規劃對既有正確步驟與資料流程造成干擾的可能性。

3. 建構具備自我修正能力之多步驟 API 任務代理人

本研究整合經驗重用與動態局部修復機制，建構 CBR-APIKGAgent，使代理人能於規劃、執行、錯誤偵測、經驗檢索、局部修復與重新規劃之間形成完整的自我修正流程。藉由此架構，本研究期望提升代理人在多步驟 API 任務中的任務完成能力與失敗恢復能力。

4. 探討經驗重用與動態局部修復機制之影響

本研究進一步探討經驗重用與動態局部修復機制對代理人自我修正能力之影響，並分析其於不同錯誤情境下的適用性與限制。透過 AppWorld [3] 多應用程式 API 任務環境進行評估，本研究期望釐清上述機制對任務完成率與錯誤修復表現的實際影響。

綜合而言，本研究旨在驗證：結合由訓練資料成功任務軌跡所萃取之全域經驗與動態局部計畫修復，是否能強化大型語言模型代理人在多步驟 API 任務中的自我修正能力，並進一步釐清兩項機制於不同錯誤情境下的效益與能力邊界。

1.3 論文組織架構

本論文共分為六章，各章內容概述如下：

- **第一章緒論：**說明研究背景、研究動機與研究目的，並概述本論文的整體章節安排。
- **第二章相關技術與文獻探討：**整理大型語言模型代理人、API 任務自動化、案例式學習與經驗重用、動態計畫修復，以及 APIKGAgent 等相關技術與研究。
- **第三章 CBR-APIKGAgent 系統架構與方法：**介紹 CBR-APIKGAgent 的整體架構、原子化經驗重用機制、動態局部修復機制，以及系統的整合運作流程。
- **第四章系統實作：**說明開發環境、系統實作流程，以及原子化經驗建立、語意檢索、提示注入與局部修復模組等實作細節。
- **第五章實驗與結果分析：**說明研究問題、實驗設置與評估指標，並透過整體效能比較、消融實驗及動態局部修復分析，評估本研究方法對任務完成率與錯誤修復表現的影響。
- **第六章結論與未來研究：**總結研究成果、研究貢獻與限制，並提出修復經驗累積、多輪局部修復、約束感知修復及跨基準驗證等未來研究方向。

第二章 相關技術與文獻探討

2.1 大型語言模型代理人

大型語言模型代理人 (Large Language Model Agent, LLM Agent) 係指以大型語言模型 (Large Language Model, LLM) 作為核心決策元件，並結合外部工具、記憶機制與環境互動能力之智慧代理系統。相較於傳統大型語言模型僅能根據輸入文字產生回應，LLM Agent 能夠根據任務目標進行推理、規劃、工具呼叫以及結果反思，使其具備執行複雜任務的能力 [1]。

近年來，隨著 GPT-4、Claude 等大型語言模型能力提升，研究者開始將其應用於任務導向情境，透過整合工具使用 (Tool Use)、任務規劃 (Task Planning)、記憶管理 (Memory Management) 與行動執行 (Action Execution) 等能力，使代理人能夠完成資訊搜尋、資料分析、程式生成以及多步驟任務自動化等工作。因此，LLM Agent 已逐漸成為大型語言模型應用的重要發展方向之一。

2.1.1 大型語言模型代理人概述

傳統大型語言模型主要透過文字輸入與輸出進行互動，其知識來源侷限於訓練資料與提示內容，因此難以取得即時資訊或操作外部系統。為克服此限制，LLM Agent 將大型語言模型視為決策核心，並透過外部工具擴展其能力，使其能夠感知環境、進行推理並執行實際操作。

Wang 等人 [1] 將 LLM Agent 描述為由感知 (Perception)、推理 (Reasoning) 與行動 (Action) 三個主要能力所組成。其中，感知能力負責理解使用者需求與環境資訊；推理能力負責任務分析、規劃與決策；而行動能力則透過工具或外部系統完成實際操作。此架構使代理人能夠將複雜任務拆解為多個子任務，並根據執行結果持續調整後續行為。

此外，部分研究進一步加入記憶 (Memory) 機制，使代理人能夠保存過往任務資訊與執行結果，提升長期任務執行能力與決策品質 [4]。透過結合記憶、規劃與工具使用能力，LLM Agent 已逐漸由單純的對話系統演變為能夠自主完成任務的智慧代理人。

2.1.2 工具使用與任務規劃

由於大型語言模型本身無法直接存取外部系統，因此工具使用 (Tool Use) 已成為現代 LLM Agent 的核心能力之一。工具使用係指代理人透過 API、函式呼叫 (Function Calling) 或其他外部工具介面執行特定操作，例如資料查詢、網頁搜尋、資料庫存取或檔案處理等。

Yao 等人提出 ReAct (Reasoning and Acting) 方法，將推理 (Reasoning) 與行動 (Acting) 整合於同一流程中，使大型語言模型能夠交替進行思考與工具操作 [5]。

ReAct 透過推理過程產生下一步行動，並根據工具執行結果更新後續決策，有效提升代理人在複雜任務中的問題解決能力。

然而，當任務規模增加時，僅依賴逐步推理可能導致規劃不完整或錯誤累積。因此，研究者開始探討任務規劃（Task Planning）機制，使代理人能夠先建立整體任務計畫，再依序執行各項步驟。Wang 等人提出 Plan-and-Solve 方法，透過先規劃後執行（Plan-then-Solve）的策略，降低大型語言模型直接推理時可能產生的遺漏步驟問題 [6]。

對於多步驟任務而言，規劃能力不僅涉及任務拆解，亦包含步驟間資料依賴關係管理與執行順序安排。當任務需要呼叫多個 API 或整合不同工具時，代理人必須確保前一階段產生之結果能夠正確傳遞至後續步驟，否則可能導致任務失敗。因此，工具使用與任務規劃能力已成為現代 LLM Agent 架構的重要組成部分。

2.1.3 大型語言模型代理人相關研究

隨著 LLM Agent 發展，研究者開始探討如何提升代理人在複雜任務中的決策能力、自我修正能力以及長期任務執行能力。

ReAct [5] 將推理與工具操作整合於單一流程中，使代理人能夠根據工具執行結果持續更新推理過程，為後續代理人研究奠定基礎。然而，ReAct 主要採用逐步決策方式，缺乏全域任務規劃能力，因此在長流程任務中可能產生錯誤累積問題。

為改善大型語言模型的規劃能力，Plan-and-Solve [6] 提出先規劃後執行的架構，使模型在執行前先建立完整任務計畫，以降低遺漏步驟與推理錯誤。然而，此類方法仍高度依賴初始規劃品質，當執行過程發生錯誤時，往往缺乏有效的修正機制。

在自我修正方面，Shinn 等人提出 Reflexion 架構，透過語言回饋（Verbal Feedback）機制使代理人能夠根據過往失敗經驗進行反思與修正 [7]。該方法利用執行結果作為回饋訊號，協助代理人改善後續決策品質，展現出大型語言模型具備一定程度自我修正能力的可能性。

此外，Park 等人提出 Generative Agents 架構，透過記憶儲存、檢索與反思機制，使代理人能夠根據歷史經驗調整後續行為 [4]。此研究顯示記憶與經驗重用對於提升代理人長期決策能力具有重要影響。

綜合上述研究可以發現，目前 LLM Agent 已具備工具使用、任務規劃、自我修正與記憶管理等能力。然而，當代理人面對多步驟 API 任務時，仍可能遭遇規劃錯誤、執行失敗以及錯誤修復效率不足等問題。因此，如何結合經驗重用與動態局部修復機制以提升代理人在複雜任務中的自我修正能力，仍是值得進一步探討的重要研究方向。

2.2 API 任務自動化

近年來，隨著大型語言模型代理人逐漸具備工具使用能力，研究者開始探討如何透過應用程式介面（Application Programming Interface, API）使代理人能夠與外部系統互動，進而完成實際任務。相較於傳統僅產生文字回應的大型語言模型，具備 API 呼

叫能力的代理人可執行資料查詢、訊息傳送、檔案管理、行事曆安排及電子商務等操作，使大型語言模型由資訊提供者轉變為任務執行者。因此，API 任務自動化已成為大型語言模型代理人研究的重要方向之一。

2.2.1 API 任務特性

API 任務係指代理人透過呼叫一個或多個 API 完成特定目標之過程。與一般問答任務不同，API 任務通常涉及外部環境狀態變化，因此代理人不僅需要產生正確答案，更必須透過實際操作使系統狀態符合使用者需求。

在簡單情況下，任務可能僅需單次 API 呼叫即可完成。然而，在真實應用情境中，多數任務通常包含多個相互依賴的步驟。例如，代理人可能需要先搜尋符合條件的使用者，再取得詳細資訊，最後執行修改或刪除操作。此類任務不僅涉及 API 選擇問題，也包含資料依賴管理、參數建構與執行順序規劃等挑戰。

此外，API 任務通常具有狀態性（Statefulness）特性。代理人在執行過程中所產生的結果將影響後續步驟的執行條件，因此必須正確保存與使用中間資料。若任一步驟發生錯誤，錯誤可能沿著後續流程持續傳播，最終導致整體任務失敗。此特性使 API 任務相較於一般文字推理任務更具挑戰性。

2.2.2 API 任務自動化相關研究

為提升大型語言模型於工具使用與 API 操作情境中的能力，研究者提出多種 API 任務自動化方法。

ToolBench [8] 建立大規模工具學習資料集，涵蓋大量真實 API 與工具操作情境，並透過工具呼叫軌跡訓練大型語言模型，使其能夠學習工具選擇與參數使用方式。此研究顯示大型語言模型可透過工具使用資料提升外部工具操作能力。

ToolLLM [9] 進一步探討大型語言模型於大規模 API 環境中的工具使用能力。該研究蒐集超過一萬六千個真實 API，並提出 API 檢索與工具使用流程，使代理人能夠從大量候選工具中選擇適當 API 完成任務。研究結果顯示，當 API 數量增加時，如何有效搜尋與選擇工具將成為影響代理人表現的重要因素。

然而，多數 API 任務研究主要聚焦於工具選擇與 API 呼叫能力，較少考量長流程任務中的規劃、錯誤修復與經驗重用問題。當任務涉及多個 API、跨應用程式操作或複雜資料依賴關係時，代理人仍可能因規劃錯誤、資料遺失或執行失敗而無法完成任務。因此，如何提升代理人在複雜多步驟 API 任務中的執行能力，仍是重要研究議題。

2.2.3 AppWorld 基準環境

為評估大型語言模型代理人於真實應用情境中的任務執行能力，Trivedi 等人提出 AppWorld 基準環境（Benchmark Environment）[3]。AppWorld 建立一個模擬多應用程式

生態系統，包含電子郵件、行事曆、購物、銀行、社群媒體等多種應用服務，並提供數百個 API 供代理人操作。

與傳統問答資料集不同，AppWorld 採用可執行環境（Executable Environment）作為評估基礎。代理人必須透過 API 呼叫實際改變環境狀態，並達成指定任務目標，才能被視為成功完成任務。因此，評估結果不僅取決於文字輸出是否正確，更取決於最終環境狀態是否符合任務需求。

此外，AppWorld 所設計之任務多為跨應用程式、多步驟任務。例如，代理人可能需要先查詢電子郵件內容，再根據資訊建立行事曆事件，或透過多個 API 完成資料搜尋與更新操作。此類任務能夠有效測試代理人的規劃能力、工具使用能力以及錯誤修復能力，因此已逐漸成為評估大型語言模型代理人的重要基準環境。

由於本研究所探討之問題涉及多步驟 API 任務中的經驗重用與動態局部修復機制，因此選擇 AppWorld 作為主要實驗平台，以驗證所提出方法於複雜 API 任務環境中的實際效益。

2.3 案例式學習與經驗重用

案例式學習（Case-Based Learning）係利用既有案例所包含的問題情境、處理策略與執行結果，協助系統處理新的相似問題。此類方法的核心並非直接複製過往解答，而是先判斷新舊情境的相似性，再選擇可重用的知識並依目前條件調整。對大型語言模型代理人而言，任務軌跡記錄了任務目標、規劃步驟、工具呼叫、資料依賴與執行結果，因而可作為經驗知識的重要來源。然而，完整軌跡通常包含大量僅適用於單一任務的實體、參數與中間資料，若未經整理便直接提供給代理人，可能增加提示內容並引入與當前任務無關的資訊。因此，如何將任務軌跡轉換為適當粒度的經驗，並在需要時檢索相關內容，是經驗重用機制的重要課題。

2.3.1 案例式推理

案例式推理（Case-Based Reasoning, CBR）是一種以過往問題求解經驗處理新問題的方法。Aamodt 與 Plaza 將其概括為檢索（Retrieve）、重用（Reuse）、修訂（Revise）與保留（Retain）四個階段所構成的循環 [10]。檢索階段從案例庫中找出與當前問題相似的案例；重用階段將既有案例的解法套用或調整至新問題；修訂階段根據實際結果檢查並修正解法；保留階段則將新的求解經驗整理後存回案例庫，供後續問題使用。

CBR 的案例通常由問題描述、解決方案及結果等資訊組成，而案例是否能有效重用，取決於情境表示、相似性判斷與解法調整方式。若案例表示過於具體，檢索結果可能只適用於幾乎相同的任務；若抽象程度過高，則可能失去執行時所需的條件與限制。因此，案例設計需在可重用性與情境完整性之間取得平衡。

將 CBR 概念應用於多步驟 API 任務時，可將使用者目標、任務結構與資料依賴視為問題情境，將規劃原則、必要中間資訊與操作限制視為可重用解法。由於不同任務

所使用的應用程式、實體名稱與參數值可能不同，直接複製完整成功計畫未必適用於新任務；相較之下，從成功軌跡萃取適用條件與策略原則，較能保留跨任務重用的可能性。此觀點構成本研究將成功任務軌跡整理為原子化經驗的概念基礎。

2.3.2 經驗重用相關研究

在大型語言模型代理人研究中，經驗通常以自然語言反思、記憶片段、完整任務軌跡或抽象策略等形式保存。Park 等人提出 Generative Agents，將代理人的觀察保存於記憶串流，並根據時近性、重要性與相關性檢索記憶，同時透過反思產生較高層次的概念 [4]。此研究說明外部記憶不僅可保存歷史事件，也能透過檢索與整理影響後續規劃。

Reflexion 則利用文字回饋建立代理人的反思記憶，使代理人在不更新模型參數的情況下，根據先前嘗試的結果調整後續決策 [7]。此方法顯示自然語言形式的經驗可作為代理人的決策提示，但其經驗主要來自代理人先前的試誤過程，與預先由訓練資料建立經驗庫的設定有所不同。

Zhao 等人提出 ExpeL，讓代理人從一組訓練任務的執行經驗中萃取自然語言知識，並於推論階段回想相關洞見與過往軌跡，以協助新的任務決策 [11]。此方法與 CBR 皆強調從既有任務經驗尋找可轉移知識，也說明代理人可藉由外部經驗庫取得參考，而不必修改大型語言模型的參數。

上述研究顯示，經驗重用大致可分為保存原始軌跡與萃取抽象知識兩種方向。原始軌跡保留較完整的行動與觀察資訊，但可能包含任務特定內容與較多雜訊；抽象知識較容易跨任務使用，但若缺少適用情境、必要中間資訊或常見陷阱，也可能被錯誤套用。因此，本研究從訓練資料中的成功任務證據離線萃取結構化原子化經驗，將完整任務中可跨任務重用的策略知識整理為較細粒度的經驗單元，並保留其適用情境、核心原則、必要中間概念與常見陷阱。這些經驗提供給代理人的規劃、重新規劃與局部修復流程作為策略參考，而不是直接複製來源任務的完整解題步驟。

需特別說明的是，本研究主要借用 CBR 與經驗重用研究中的檢索與重用概念，正式經驗庫由訓練資料中的成功任務軌跡離線建立；測試期間產生的修復經驗不會寫回正式檢索經驗庫，因此本研究不涉及測試期間的線上經驗累積、持續學習或完整 CBR Retain 循環。

2.3.3 語意檢索方法

經驗庫規模增加後，系統需要從大量經驗中選出與當前情境相關的內容。傳統關鍵字檢索主要依賴詞彙重疊，當任務與經驗使用不同描述方式時，可能無法找出語意相近的案例。語意檢索則將查詢與候選內容轉換為稠密向量，並在向量空間中計算相似程度。Karpukhin 等人提出 Dense Passage Retrieval，利用雙編碼器分別建立查詢與文本的稠密表示，說明稠密向量可用於找出語意相關而不僅是詞彙相同的內容 [12]。

Lewis 等人提出檢索增強生成 (Retrieval-Augmented Generation, RAG)，將預訓練生成模型視為參數化記憶，並以可檢索的稠密向量索引作為非參數化記憶，使模型能依據查詢取得外部內容後再進行生成 [13]。此概念使知識可獨立於模型參數保存與更新，也適合用於將任務經驗提供給大型語言模型代理人參考。

語意檢索常使用餘弦相似度衡量查詢向量 $\mathbf{q}$ 與經驗向量 $\mathbf{e}$ 的接近程度，其定義如下：

$$\text{sim}(\mathbf{q}, \mathbf{e}) = \frac{\mathbf{q} \cdot \mathbf{e}}{\|\mathbf{q}\| \|\mathbf{e}\|}. \quad (2.1)$$

系統可依相似度排序並取得前 k 筆經驗，再將其加入代理人的提示內容。然而，語意相似並不代表策略必然適用；若查詢內容未包含當前階段的目標、失敗資訊或資料依賴，仍可能檢索到表面相似但無法使用的經驗。此外，檢索數量過多也可能增加無關資訊，干擾模型原有判斷。因此，經驗內容的結構、查詢資訊的選擇與檢索結果的注入方式，皆會影響經驗重用的實際表現。

依據上述概念，本研究以語意檢索從訓練資料建立的原子化經驗庫中取得與當前任務或失敗情境相關的內容，並分別提供給初始規劃、重新規劃與局部修復流程。由於三個階段所面對的資訊不同，檢索時需考量任務目標、既有計畫、失敗步驟與錯誤資訊等情境，使取得的經驗能對應目前的決策需求。檢索到的內容僅作為策略參考，而非代理人必須遵循的固定規則；此機制是否有助於多步驟 API 任務的規劃與修復，仍需透過後續實驗加以評估。至於原子化經驗的欄位設計、索引建立方式與各階段的查詢及注入流程，將於第三章與第四章進一步說明。

2.4 動態計畫修復與自我修正

大型語言模型代理人在多步驟任務中通常需要先產生計畫，再依序呼叫工具或 API 完成任務。然而，在實際執行過程中，代理人可能因任務理解錯誤、API 選擇不當、參數推論錯誤、資料依賴缺失或中間結果不符合預期而導致任務失敗。此類錯誤若未能即時處理，往往會沿著後續步驟傳遞，形成錯誤傳播 (Error Propagation) 問題，使後續計畫即使本身合理，也可能因依賴錯誤資料而無法成功執行。

因此，近年大型語言模型代理人研究逐漸重視執行回饋、重新規劃與自我修正能力。相較於一次性產生完整答案或完整計畫，具備修正能力的代理人能夠根據環境回饋、工具執行結果或錯誤訊息調整後續行動。此類方法顯示，代理人不僅需要初始規劃能力，也需要在執行過程中偵測錯誤、理解失敗原因，並根據當前狀態產生新的行動策略。

2.4.1 重新規劃機制

重新規劃 (Replanning) 是多步驟代理人常見的錯誤處理方法。當代理人在執行過程中遇到錯誤或原始計畫無法繼續完成時，系統會根據目前狀態、已完成步驟、失敗

資訊與任務目標重新產生後續計畫。此機制使代理人能夠在初始計畫不完整或環境狀態發生變化時，重新調整任務執行方向。

在大型語言模型代理人中，重新規劃通常與工具執行回饋結合。ReAct 將推理與行動交替進行，使模型能夠根據工具回傳結果更新後續推理與行動決策 [5]。此類方法展現了代理人根據執行觀察調整行為的能力，也為後續具備執行回饋與重新決策能力的代理人架構奠定基礎。

然而，對於多步驟 API 任務而言，完全重新規劃並非總是最適合的修復方式。當錯誤僅發生於局部步驟，例如缺少某個中間資料、API 參數錯誤，或單一步驟使用了不適合的 API 時，若直接重新產生整體計畫，可能使原本已正確完成的步驟與資料流程被改寫，進而引入新的不確定性。此外，API 任務通常具有明確的資料依賴關係，前一步驟取得的識別碼、查詢結果或中間狀態，常會影響後續 API 呼叫。因此，重新規劃雖然能提供較大範圍的修正能力，但也可能造成不必要的整體變動。

2.4.2 自我修正機制

自我修正 (Self-Correction) 是指大型語言模型或代理人根據自身輸出、外部回饋或執行結果，對原始答案、推理過程或行動計畫進行檢查與修改的能力。此類方法通常不需要更新模型參數，而是透過提示設計、回饋訊號或記憶機制，使模型在推論階段改善原始輸出。

Reflexion 提出以語言回饋作為代理人自我改進的方式，使代理人能夠根據任務失敗訊號產生反思內容，並將其作為後續嘗試的決策依據 [7]。此方法顯示，文字形式的失敗分析與經驗描述可以在不進行模型微調的情況下，輔助代理人改善後續行動。

Self-Refine 則進一步展示大型語言模型可透過自我回饋與反覆修訂流程改善初始輸出 [14]。其核心概念為先產生結果，再由模型自行分析缺失並提出修改建議，最後依據回饋重新生成內容。研究結果顯示，即使不進行額外訓練，模型仍能透過反覆修正提升輸出品質。

除了直接產生反思或修訂內容外，近期亦有研究從錯誤分析角度探討大型語言模型代理人的失敗原因。Zhu 等人提出 AgentErrorTaxonomy，將代理人失敗歸納為記憶、反思、規劃、行動與系統等面向，並進一步提出 AgentDebug 以定位造成任務失敗的根本錯誤並產生修正回饋 [15]。此類研究顯示，將失敗軌跡轉換為結構化錯誤描述與修正建議，可作為代理人後續自我修正的重要依據。不過，根因定位通常需要明確的錯誤標註、可分析的失敗軌跡或額外除錯程序；若缺乏可驗證的根因資料，錯誤分類更適合作為修正流程中的輔助訊號，而非直接等同於完整的根因診斷。

在工具使用與 API 任務場景中，自我修正的回饋來源不僅包含模型自身判斷，也包含 API 回傳結果、錯誤訊息、執行紀錄與中間資料狀態。這些外部訊號能提供比純文字生成任務更具體的錯誤線索。例如，API 呼叫失敗可能指出缺少必要參數，查詢結果為空可能表示前置搜尋條件不正確，而後續步驟無法執行則可能反映資料依賴尚未建立。若代理人能夠利用這些訊號分析失敗原因，便有機會針對錯誤位置進行修正，而非完全依賴重新產生整體計畫。

2.4.3 動態計畫修復相關研究

動態計畫修復 (Dynamic Plan Repair) 關注代理人在執行過程中面對錯誤、環境變化或計畫不可行時，如何根據當前狀態修正原有計畫。相較於重新產生完整計畫，計畫修復更強調保留原計畫中仍然有效的部分，並針對失敗區段進行局部調整。

近年研究開始探討如何結合驗證與修復機制提升代理人執行能力。DVR (Divide-Verify-Refine) 提出將複雜任務拆解後逐步驗證，並根據驗證結果進行修正的流程 [16]。其核心概念在於利用外部驗證訊號找出失敗原因，再透過修正程序改善後續執行結果。若與前述 Reflexion 及 Self-Refine 共同觀察，可發現這些研究雖然採用不同形式的回饋來源與修正流程，但皆支持一項共同觀點：大型語言模型代理人可利用失敗訊號、驗證結果或自我回饋，對原始推理或行動計畫進行後續修正。此類方法顯示，相較於直接重新產生完整解答，利用失敗資訊進行針對性修正可能有助於改善代理人的錯誤修復表現。

對 API 任務而言，動態計畫修復具有特別的重要性。API 任務中的每個步驟通常不只是單一動作，而是會產生後續步驟所需的資料，例如物件識別碼、查詢結果、時間條件、使用者資訊或狀態更新結果。因此，局部錯誤可能導致後續步驟缺少必要輸入，進而造成連鎖失敗。若代理人能在錯誤發生後利用失敗資訊與相依資料，針對局部計畫進行插入、替換或刪除，便能在不完全推翻原計畫的情況下嘗試恢復任務執行。

綜合上述研究可以發現，重新規劃著重於重新建立任務執行流程，自我修正強調根據回饋改善決策品質，而動態計畫修復則介於兩者之間，試圖利用執行過程中的錯誤資訊修正局部計畫。此概念亦成為本研究設計動態局部修復機制的重要基礎。

2.5 APIKGAgent

APIKGAgent (API Knowledge Graph Agent) 為吳柏諭所提出之大型語言模型代理人架構，其核心概念為結合 API 知識圖譜 (API Knowledge Graph) 與大型語言模型，使代理人能夠在大量 API 環境中完成多步驟任務 [2]。相較於直接將完整 API 文件提供給大型語言模型進行推理，APIKGAgent 透過知識圖譜保存 API、參數以及參數依賴關係，並利用圖譜搜尋結果輔助任務規劃與執行，以降低大型語言模型於 API 檢索與參數推理過程中的錯誤。

2.5.1 APIKGAgent 架構概述

APIKGAgent 的整體流程可分為 API 搜尋、任務規劃、任務執行與重新規劃四個階段，其架構如圖 2.1 所示 [2]。

其主要流程可整理如下：

- **API 搜尋**：系統根據使用者任務描述擷取關鍵資訊，並透過 API 知識圖譜搜尋可能相關之 API。由於知識圖譜中保存了 API 與參數之間的依賴關係，因此系統能

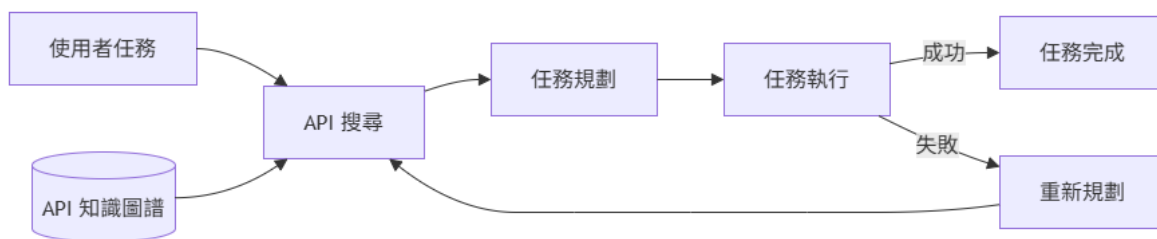


圖 2.1: APIKGAgent 系統架構圖（改繪自 [2]）

夠縮小搜尋範圍，避免大型語言模型直接面對大量 API 文件所造成的推理負擔。

- **任務規劃：**完成 API 搜尋後，任務規劃模組會根據使用者需求與候選 API 集合產生任務計畫（Task Plan）。系統將複雜任務拆解為多個步驟，並決定各步驟所需之 API 呼叫順序與資料流向，使後續執行流程能夠依照規劃逐步完成任務。
- **任務執行：**在任務執行階段，代理人依照規劃結果逐步執行 API 呼叫與資料處理流程。執行過程中所產生的中間結果會被保存，供後續步驟使用，以維持多步驟任務中的資料依賴關係。此外，系統亦會記錄各步驟之執行結果，作為後續錯誤分析與重新規劃的依據。
- **重新規劃：**當任務執行失敗或無法繼續完成時，系統則啟動重新規劃機制。重新規劃模組會根據目前執行狀態、錯誤資訊以及已完成步驟重新產生後續計畫，使代理人能夠在部分步驟失敗的情況下持續嘗試完成任務。

透過 API 知識圖譜、任務規劃、任務執行以及重新規劃等模組的整合，APIKGAgent 建立了一套完整的多步驟 API 任務執行流程，並驗證了知識圖譜輔助 API 搜尋與規劃機制在複雜 API 任務中的可行性。

2.5.2 APIKGAgent 之研究缺口

雖然 APIKGAgent 已能透過知識圖譜改善 API 搜尋效率，並利用重新規劃機制提升多步驟任務之完成能力，但在實際任務執行過程中仍存在若干限制。

首先，當任務執行失敗時，重新規劃機制主要依賴大型語言模型根據當前執行狀態重新產生計畫。然而，本研究接續之基礎版本尚未將成功軌跡整理為可檢索、可重用的經驗庫，因此當面對相似問題時，代理人缺乏可供參考的跨任務策略知識，仍可能重複產生相同類型的規劃錯誤或執行錯誤。換言之，系統尚未系統化利用跨任務經驗重用（Experience Reuse）輔助規劃與修復。

其次，APIKGAgent 的錯誤修復主要依賴重新規劃機制處理執行失敗情況。然而，在多步驟任務中，部分錯誤可能僅發生於局部步驟，例如遺漏必要資訊蒐集步驟、錯誤使用中間資料，或選擇不適當的 API 等情況。此類錯誤未必需要重新產生整體計畫即可修正，但現有架構缺乏針對局部錯誤進行修復的能力。

此外，重新規劃結果高度依賴大型語言模型於當下的推理能力，缺乏可供參考之成功軌跡策略經驗與修復策略。因此，當任務結構較為複雜或錯誤原因較難辨識時，系統仍可能出現重複失敗、修復方向偏離原始任務目標，或重新規劃後仍無法有效解決問題等情況。

綜合上述分析可以發現，現有 APIKGAgent 雖已具備 API 搜尋、任務規劃、執行監控與重新規劃等能力，但在經驗重用與局部錯誤修復方面仍有進一步改善空間。因此，本研究提出案例式修復 API 知識圖譜代理人（Case-Based Repair APIKGAgent, CBR-APIKGAgent），透過原子化經驗（Atomic Experience）重用機制與動態局部修復機制，探討上述機制是否能提升代理人在多步驟 API 任務中的自我修正能力與任務完成率。



第三章 CBR-APIKGAgent 系統架構與方法

本章說明本研究所提出之 CBR-APIKGAgent 的系統架構與核心方法。針對多步驟 API 任務中的資料依賴、錯誤傳播與局部失敗修復問題，本研究設計兩項核心機制：其一為由訓練資料成功任務軌跡建立的原子化經驗重用機制，其二為針對執行失敗進行局部計畫調整的動態局部修復機制。以下首先介紹整體系統架構與任務流程，接著分別說明原子化經驗的概念、檢索與使用方式，以及局部修復如何根據失敗資訊進行插入、替換或刪除等計畫調整。

3.1 CBR-APIKGAgent 系統總覽

CBR-APIKGAgent 的整體架構以多步驟 API 代理人的典型執行流程為主軸，包含 API 搜尋、任務規劃、步驟式執行與失敗後重新規劃等階段。在此流程中，本研究進一步配置原子化經驗庫與動態局部修復機制，使系統能在規劃、重新規劃與局部修復時取得案例式策略參考。以下先以架構觀點整理兩項核心機制在系統中的角色，再以演算法形式說明完整任務執行流程。

3.1.1 系統架構與核心機制

CBR-APIKGAgent 在多步驟 API 代理人架構中加入兩項核心機制：原子化經驗重用與動態局部修復。前者提供規劃與修復時的案例式策略參考，後者在步驟失敗後嘗試以局部計畫修改恢復執行。此設計使代理人在面對相似任務結構或局部執行失敗時，能取得來自訓練資料成功任務軌跡的策略參考，而不僅依賴當下錯誤訊息重新產生後續動作。

在整體架構中，API 搜尋、任務規劃與任務執行構成基本任務流程；原子化經驗檢索器支援初始規劃、重新規劃與局部修復三個決策點；局部修復模組則在步驟失敗後嘗試小範圍計畫修改，並於無法形成有效修改時交由重新規劃處理。

案例式經驗檢索與動態局部修復雖然在任務失敗時會互相配合，但兩者在系統中扮演不同角色。案例式經驗檢索負責提供與當前任務或錯誤情境相關的策略參考，屬於決策輔助機制；動態局部修復則負責根據失敗資訊修改局部計畫，屬於錯誤處理機

[illegible]

為了使後續方法說明能聚焦於本研究提出的核心機制，表 3.1 以兩項設計為主軸，整理原子化經驗重用與動態局部修復機制在整體架構中的角色。

核心設計	架構角色	核心作用
原子化經驗重用	決策輔助層	由原子化經驗庫與語意檢索流程提供任務結構、資料依賴、常見陷阱與修復策略等參考。
動態局部修復	錯誤處理層	在步驟失敗後整理失敗上下文，結合候選修復 API 與修復經驗嘗試修改局部計畫；若無法形成有效修改，則回退至重新規劃。

3.1.2 整體任務執行演算法

15

演算法 1: CBR-APIKGAgent 整體任務執行演算法

Input: user task T , API knowledge graph G , atomic experience base $\mathcal{E}$

Output: execution result R

```
1  $C_{api} \leftarrow \text{SearchAPIs}(G, T);$ 
2  $\mathcal{E}_p \leftarrow \text{RetrievePlanningExperience}(\mathcal{E}, T);$ 
3  $P \leftarrow \text{GeneratePlan}(T, C_{api}, \mathcal{E}_p);$ 
4 while HasNextStep( $P$ ) do
5    $s \leftarrow \text{NextStep}(P);$ 
6    $o \leftarrow \text{ExecuteStep}(s);$ 
7   if IsSuccess( $o$ ) then
8     UpdateState( $s, o$ );
9   else
10     $F \leftarrow \text{CollectFailureContext}(s, o);$ 
11     $(c, f) \leftarrow \text{AnalyzeFailure}(F);$ 
12     $P_{\text{repair}} \leftarrow \emptyset;$ 
13    if IsRepairAllowed( $c, F$ ) then
14       $C_{\text{repair}} \leftarrow \text{SearchRepairAPIs}(G, F, f);$ 
15       $\mathcal{E}_r \leftarrow \text{RetrieveRepairExperience}(\mathcal{E}, F);$ 
16       $m \leftarrow \text{SelectRepairMode}(F, C_{\text{repair}}, \mathcal{E}_r);$ 
17      if  $m \neq \emptyset$  then
18         $P_{\text{repair}} \leftarrow \text{GenerateRepairPlan}(P, s, m);$ 
19    if  $P_{\text{repair}} \neq \emptyset$  then
20       $P \leftarrow P_{\text{repair}};$ 
21      SetNextStepToRepairPoint( $P$ );
22    else
23       $\mathcal{E}_{rp} \leftarrow \text{RetrieveReplanningExperience}(\mathcal{E}, T, P, F);$ 
24       $P \leftarrow \text{Replan}(T, P, F, \mathcal{E}_{rp});$ 
25 return  $R$ 
```

演算法 1 的詳細步驟說明如下。為了便於對照演算法結構，以下依照初始化階段、主要執行迴圈與結果回傳三個部分說明。

1. **初始化階段：建立初始計畫。**系統接收使用者任務 T 後，先根據 API 知識圖譜 G 與原子化經驗庫 $\mathcal{E}$ 建立可執行的初始計畫 P ，作為後續逐步執行的計畫依據。

- **API 搜尋：**系統以 T 作為查詢目標，於 G 中搜尋與任務相關的 API，形成初始候選 API 集合 C_{api} 。
- **規劃經驗檢索：**系統同時以 T 作為經驗查詢脈絡，從 $\mathcal{E}$ 中取得與任務結構、資料依賴或操作順序相關的規劃導向經驗 $\mathcal{E}_p$ 。
- **初始計畫生成：**任務規劃器將 T 、 C_{api} 與 $\mathcal{E}_p$ 作為輸入，產生初始計畫 P 。此計畫包含 API 呼叫、資料處理與步驟相依關係。

2. **主要執行迴圈：執行步驟並處理失敗。**產生初始計畫 P 後，系統進入主要執行迴圈。每次迭代會從目前計畫 P 中取出待執行步驟 s ，執行後得到步驟執行結果 o ；系統再依 o 判斷是否更新狀態或進入錯誤處理流程。

- **步驟執行：**系統從目前計畫 P 中取出下一個尚未完成的步驟 s ，並依 s 的內容執行 API 呼叫或資料處理動作。若 s 需要使用前面 API 回傳的資料，執行器會根據 P 中的相依關係取得對應中間結果，再產生執行結果 o 。
- **成功狀態更新：**若 o 表示步驟執行成功，系統會將 s 的輸出寫回任務狀態，供後續相依步驟使用。
- **失敗脈絡整理：**若 o 表示步驟執行失敗，系統會將 s 、 o 、錯誤訊息、API 回饋與相關相依資料整理為失敗脈絡 F ，供後續錯誤分類、局部修復與重新規劃使用。
- **錯誤分類與可行回饋：**系統根據 F 產生暫定錯誤分類 c 與可行回饋 f ，前者用於輔助錯誤處理路由，後者用於描述可能採取的修復方向。
- **局部修復嘗試：**若 c 與 F 顯示仍可嘗試局部修復，系統會根據 F 與 f 搜尋可能有助於修復的 API，形成修復候選 API 集合 C_{repair} ，並從 $\mathcal{E}$ 中檢索與目前失敗情境相關的修復導向經驗 $\mathcal{E}_r$ 。
- **修復策略選擇：**系統將 F 、 C_{repair} 與 $\mathcal{E}_r$ 作為修復策略判斷的主要脈絡，決定局部修改方式 m ，例如插入、替換或刪除等計畫修改方式。
- **產生修復後計畫：**若 m 不為空，系統會依據局部修改方式 m 對目前計畫 P 進行局部修改，產生修復後計畫 P_{repair} 。此階段只改寫計畫內容，不直接執行新增或替換後的 API 步驟。例如，系統可在缺少前置資料時插入查詢步驟，於原步驟 API 不適用時替換步驟，或在步驟重複時刪除該步驟。
- **設定後續執行位置：**若 P_{repair} 成功產生，系統會以 P_{repair} 更新 P ，並將後續執行位置調整至修復後計畫中需要接續執行的位置。若採用插入或替換策略，下一次取出的步驟會是新插入或替換後的修復步驟；若採用刪除策略，下一次取出的步驟則會是刪除後補位至該位置的後續步驟。
- **重新規劃回退：**若系統未形成 P_{repair} ，則檢索重新規劃相關經驗 $\mathcal{E}_{\text{tp}}$ 。系統接著將 T 、既有計畫 P 、失敗脈絡 F 與 $\mathcal{E}_{\text{tp}}$ 交由重新規劃器產生後續計畫。重

新產生的計畫會更新 P ，再交回任務執行流程，使系統繼續執行尚未完成的任務。

3. **結果回傳：輸出執行結果。**當目前計畫 P 中已無尚未執行的步驟，或任務流程到達終止條件時，系統回傳最終執行結果 R 。後續實驗章節將依據 AppWorld 評估結果判斷 R 是否符合任務需求。

整體而言，CBR-APIKGAgent 採取「先局部修復、必要時重新規劃」的錯誤處理流程。此流程的設計目的並非取代重新規劃，而是讓系統在錯誤可能由小範圍計畫調整處理時，先嘗試保留原本仍有效的計畫與資料流程；當局部修復無法形成可執行修改，或修復嘗試達到限制時，再回退至重新規劃。由於本研究不假設任務環境可回復至失敗前狀態，重新規劃與局部修復皆是在目前已觀察到的執行狀態下進行。透過原子化經驗檢索與動態局部修復的結合，本研究進一步探討案例式策略參考是否有助於改善多步驟 API 任務中的規劃品質與錯誤修復表現。

3.2 原子化經驗重用機制

本節對應演算法 1 中的經驗檢索步驟，說明原子化經驗如何由訓練資料中的成功任務軌跡建立，並於任務流程中被表示、檢索與使用。此機制並非獨立執行的子系統，而是透過語意檢索與提示注入，在初始規劃、局部修復與重新規劃階段提供策略參考。

原子化經驗重用機制的目的，是將成功任務軌跡中隱含的規劃與資料流策略，轉換為可跨任務重用的經驗知識。相較於直接保存完整任務軌跡，本研究將經驗拆解為較小的策略單元，使其能提醒代理人注意相似任務中的資料依賴、操作順序與常見陷阱，而非要求代理人複製過去任務的完整步驟。以下先說明經驗來源與原子化經驗定義，再說明其不同決策階段的檢索與使用方式。

3.2.1 經驗來源與使用範圍

本研究使用的正式經驗庫由訓練資料中的成功任務軌跡離線建立，測試期間不新增經驗至正式經驗庫，也不更新正式檢索索引。此設計使經驗來源與實驗評估資料保持切割，避免測試期間產生的執行紀錄影響後續任務評估。

雖然系統在局部修復過程中可能產生新的修復軌跡，但此類軌跡不會被納入正式經驗庫。主要原因在於，單一次局部修復不一定代表整體任務成功，且其正確性需由後續資料流與最終任務結果共同判斷。因此，本研究不將測試期間產生的修復軌跡轉換為可檢索經驗，以避免未驗證內容影響後續規劃與修復決策。

在經驗萃取階段，本研究以訓練資料中已知成功的任務為來源，將成功過程中可跨任務重用的規劃原則與資料依賴模式整理為原子化經驗。由於本研究關注的是多步驟 API 任務中的規劃與修復能力，因此經驗萃取不以保留完整 API 呼叫序列為目標，而是著重於任務成功過程中可泛化的策略知識。

3.2.2 原子化經驗定義

本研究將原子化經驗定義為一個可跨任務重用的最小策略單元，用以描述特定任務情境下應保留的規劃原則、資料流模式、必要檢查與常見陷阱。換言之，本研究並非直接重用完整案例，而是將案例中的可重用策略整理為原子化經驗，作為後續檢索與提示注入的基本單位。

一筆原子化經驗通常對應到一類可重複出現的規劃需求或錯誤風險，其重點不在於記錄某次任務使用了哪些 API，而在於保留該任務成功時必須滿足的資料流與檢查原則。例如，若某類任務需要更新符合條件的多筆資料，原子化經驗關注的不是特定 API 呼叫與固定參數，而是「先建立符合條件的目標集合，再確認各目標項目的狀態後執行更新」這類可重用的資料流原則。

為明確界定原子化經驗在本研究中的角色，以下從其保存內容與使用方式說明其定位：

- **案例粒度**：原子化經驗不重播完整任務軌跡，而是抽取其中可跨任務重用的策略片段。
- **操作層級**：原子化經驗不保存固定 API 呼叫序列或參數值，而是描述資料流、檢查條件與決策原則。
- **使用方式**：部分代理人系統會將可重用能力封裝為技能、工具或子程序；本研究的原子化經驗則著重於以自然語言形式保存細粒度策略知識，並透過檢索提供給模型作為規劃與修復參考，而非作為可直接調用的執行模組。

為了使上述經驗能被檢索、排序並注入不同決策階段，本研究進一步將每筆原子化經驗表示為結構化知識。表 3.2 以概念層級整理原子化經驗所需保留的資訊面向，用以說明成功任務軌跡如何被抽象化為可檢索經驗；實際資料欄位、儲存格式與載入流程則於第四章系統實作中說明。

表 3.2: 原子化經驗主要資訊面向與用途

資訊面向	用途說明
來源脈絡	保留經驗所來自的成功任務背景，使系統能理解該經驗原先適用的任務目標與資料情境。
適用情境	描述此經驗適合被參考的任務或錯誤情境，例如目標集合建構、候選驗證、批次處理或輸出格式限制。
核心原則	保留成功任務中可跨任務重用的規劃原則或資料流不變條件。
必要中間概念	說明規劃或修復過程中應維持的中間資料、目標集合、驗證條件或操作順序。
套用方式	描述將經驗轉移至新任務時可參考的抽象步驟，作為規劃或修復時的策略提示，而非固定 API 呼叫序列。
常見陷阱	指出此類任務中容易發生的錯誤，例如過早執行寫入、忽略篩選條件、不完整取得資料或使用錯誤目標集合。

範例：原子化經驗概念示例

以一筆用於經驗萃取的訓練任務軌跡為例，該任務要求代理人在 Venmo 社交動態中，找出「昨天」且「交易參與者包含任一位兄弟姐妹」的交易，並對這些交易按讚。此任務軌跡可萃取出多筆原子化經驗；其中一筆經驗屬於寫入目標保護類型，重點在於避免代理人在尚未確認最終目標集合前執行寫入動作，其內容可整理如下：

- **適用情境**：當某個動作需要套用於大型集合中的特定子集合，且該子集合由多個彼此獨立的篩選條件共同定義時。
- **核心原則**：寫入動作只能套用於符合所有指定篩選條件的項目，以避免不必要或錯誤的資料修改。
- **必要中間概念**：初始候選項目集合、篩選條件集合，以及套用所有條件後形成的最終操作目標集合。
- **套用方式**：先從資料來源取得完整的潛在目標集合，接著根據任務需求定義所有必要篩選條件，逐一檢查每個候選項目是否符合條件，最後只對符合所有條件的項目執行寫入動作。

- **常見陷阱：**在所有篩選條件檢查完成前就執行寫入、錯誤定義或套用篩選條件、漏掉關鍵條件，或誤以為單一條件即可決定最終目標集合。

上述資訊面向使原子化經驗同時具備可追溯性、可檢索性與可注入性。整體而言，原子化經驗以較抽象的資料來源、候選集合、驗證條件、目標集合與輸出契約等概念描述可重用策略，避免過度貼近單一訓練任務。

3.2.3 經驗檢索視角與使用情境

原子化經驗雖來自同一份經驗庫，但在不同決策階段所需強調的語意重點並不相同。為使經驗能同時支援規劃與修復，本研究將經驗檢索區分為規劃導向與修復導向兩種視角：

- **規劃導向檢索：**主要強調適用情境、核心原則、套用方式與任務目標，使經驗能支援任務結構、資料依賴或操作順序相關的規劃判斷。
- **修復導向檢索：**主要強調適用情境、核心原則、套用方式與常見陷阱，使經驗能支援失敗訊息、錯誤脈絡或修復方向相關的判斷。

此設計並不代表系統維護兩套不同來源的經驗，而是使同一批原子化經驗能依決策目的凸顯不同資訊。實際索引建立、查詢格式與提示注入方式於第四章說明。

依照使用階段不同，原子化經驗主要支援以下三種決策情境：

1. 初始規劃階段

- **使用視角：**此階段主要使用規劃導向經驗。
- **設計目的：**在初始計畫生成前，提醒代理人注意相似任務中的資料流條件、目標範圍與常見陷阱，使計畫能較早納入必要中間概念與操作順序。

2. 局部修復階段

- **使用視角：**此階段主要使用修復導向經驗。
- **設計目的：**在步驟失敗後，提供與目前失敗情境相近的資料流原則與常見陷阱，輔助判斷應補入前置資料、替換失敗操作，或移除不必要的步驟。

3. 重新規劃階段

- **使用視角：**此階段同時使用規劃導向與修復導向經驗。
- **設計目的：**重新規劃需同時回應原始任務需求與已發生的失敗，因此需兼顧任務結構層面的規劃經驗，以及錯誤情境層面的修復經驗。

整體而言，原子化經驗重用機制提供的是可依不同決策情境檢索的案例式知識，而非固定規則或保證正確的修復模板。

3.3 動態局部修復機制

本節對應演算法 1 中步驟執行失敗後的錯誤處理分支，說明 CBR-APIKGAgent 如何透過局部計畫修改嘗試恢復任務執行。由於整體任務流程已於前述演算法說明，本節聚焦於局部修復的設計目標、局部修復與重新規劃之選擇原則，以及插入、替換與刪除三種計畫修改策略。

3.3.1 局部修復設計目標與適用邊界

動態局部修復的設計目標，是在任務執行過程中發生局部失敗時，優先嘗試以小範圍計畫修改恢復執行，而非立即重新產生整體計畫。此設計背後的假設是：多步驟 API 任務中的失敗不一定代表整體任務理解錯誤，有時僅是單一步驟缺少前置資料、使用了不合適的操作方式，或包含不必要的重複步驟。在此情況下，局部修復可保留已成功執行的步驟與仍有效的中間資料，並降低重新規劃對原本正確資料流程造成干擾的可能性。

由於任務執行可能已改變外部應用程式狀態，此修復流程不假設可回復至錯誤前狀態，而是依據目前可取得的失敗資訊形成局部調整方案。然而，局部修復並不適用於所有失敗情境。若失敗被暫定為不適合局部處理，或同一原始失敗點已達修復嘗試限制，系統應回退至重新規劃。換言之，局部修復在本研究中被定位為重新規劃前的局部調整機制，而非取代重新規劃的完整錯誤處理方法。

3.3.2 局部修復與重新規劃之選擇原則

局部修復與重新規劃的選擇，決定了系統在失敗發生後應保留原計畫並局部調整，或重新產生後續計畫。此選擇並非建立在精確的錯誤根因判斷上，而是根據步驟失敗後可取得的有限資訊進行保守決策。這些資訊包含失敗步驟的位置與類型、暫定錯誤分類、錯誤訊息、可行回饋，以及同一原始失敗點的修復嘗試狀態。基於上述資訊，本研究採用下列選擇原則：

- **優先保留局部修復空間：**當失敗看似侷限於單一步驟附近，例如缺少前置資料、操作方式不合適，或局部步驟結果不足以支撐後續計畫時，系統優先嘗試局部修復，以保留已成功執行的步驟與既有資料流程。
- **避免無限制反覆修復：**若修復步驟本身再次失敗，該失敗會被視為原始失敗點的延伸，而不是新的獨立失敗點。當同一原始失敗點達到修復嘗試限制時，系統停止產生新的局部修復方案。
- **保留重新規劃作為回退機制：**若錯誤被暫定為不適合局部處理，或局部修復無法形成有效計畫修改，系統回退至重新規劃。此設計使局部修復可作為重新規劃前的嘗試，而不取代重新規劃本身。

可行回饋不直接決定上述選擇，而是在進入局部修復後作為修復行動的依據。相關限制的具體紀錄方式與流程銜接於第四章說明。

3.3.3 局部計畫修復策略

在系統判斷應嘗試局部修復後，局部計畫修復策略的目標，是根據目前失敗步驟與相關錯誤資訊產生一個可套用於原計畫片段的修改方案。此修改方案以最小化計畫變動為原則，僅調整失敗步驟附近的內容，並盡可能保留已成功執行的步驟與可用中間資料。

需要注意的是，局部修復策略的輸出是修復後的計畫片段，而非立即完成 API 執行；修復步驟被加入、替換或刪除後，系統會回到對應位置接續執行更新後的計畫。

本研究將局部計畫修改方式分為插入步驟 (Insert)、替換步驟 (Replace) 與刪除步驟 (Delete) 三類。為了說明三種策略在計畫序列上的差異，令目前計畫可表示為：

$$P = \langle S_1, S_2, \dots, S_k^{fail}, \dots, S_n \rangle$$

其中， S_k^{fail} 表示目前執行失敗的步驟， R_k 表示局部修復產生的修復步驟。對應演算法 1，三種策略皆可視為 $\text{GenerateRepairPlan}(P, s, m)$ 依據目前計畫 P 、失敗步驟 s 與局部修改方式 m ，產生修復後計畫 P_{repair} 的不同形式。以下分別說明三種策略的適用情境、計畫調整方式與使用原則；實際候選 API 搜尋、策略選擇資訊組合與計畫更新流程則於第四章說明。

1. 插入步驟 (Insert)

- **適用情境：**當原步驟本身仍可能正確，但執行前缺少必要輸入或前置狀態時，系統會考慮在原失敗步驟前插入新的修復步驟。
- **計畫調整：**新增修復步驟會被安排在原失敗步驟之前，原失敗步驟仍保留於計畫中，並於修復步驟完成後再次執行。此策略可表示為：

$$P_{\text{repair}} = \langle S_1, \dots, S_{k-1}, R_k, S_k, S_{k+1}, \dots, S_n \rangle$$

其中， R_k 用於補足原步驟所需的前置資料或狀態， S_k 則代表原失敗步驟在修復後重新進入執行流程。

- **使用原則：**插入策略的重點在於保留原失敗步驟的角色。新增步驟應用於補足資料或建立前置條件；若候選 API 與原步驟具有相同操作目的，則不應以插入方式加入，以避免原步驟後續重複執行或造成副作用。

待繪圖：插入步驟示意

修復前： $S_{k-1} \rightarrow S_k^{fail} \rightarrow S_{k+1}$
 修復後： $S_{k-1} \rightarrow R_k \rightarrow S_k \rightarrow S_{k+1}$

圖 3.2: 插入步驟之局部計畫修改示意圖

案例待補：此處預計補入一個實際任務中的 *Insert* 成功案例，說明原失敗步驟、插入的修復步驟，以及修復後如何回到原步驟執行。

2. 替換步驟 (Replace)

- **適用情境：**當錯誤訊號顯示原失敗步驟本身可能使用了不適當的 API、錯誤的操作方式，或其描述無法完成原本的局部目標時，系統會考慮以新的修復步驟替換原步驟。
- **計畫調整：**新的修復步驟會取代原失敗步驟，原失敗步驟不再於後續流程中執行。此策略可表示為：

$$P_{\text{repair}} = \langle S_1, \dots, S_{k-1}, R_k, S_{k+1}, \dots, S_n \rangle$$

其中， R_k 接替原失敗步驟在計畫中的位置，並承擔其原本應完成的局部目標。

- **使用原則：**替換策略比插入策略更可能影響資料流程，因此需根據失敗證據、候選 API 功能與相依資料，確認新步驟能合理接替原步驟在計畫中的位置；若候選 API 僅能提供前置或輔助資料，則不應直接取代原步驟。

待繪圖：替換步驟示意

修復前： $S_{k-1} \rightarrow S_k^{fail} \rightarrow S_{k+1}$

修復後： $S_{k-1} \rightarrow R_k \rightarrow S_{k+1}$

圖 3.3: 替換步驟之局部計畫修改示意圖

案例待補：此處預計補入一個實際任務中的 *Replace* 成功案例，說明原失敗步驟、替換後的修復步驟，以及為何原步驟不應再次執行。

3. 刪除步驟 (Delete)

- **適用情境：**當失敗步驟被判斷為冗餘、重複或不應繼續執行時，系統會考慮將其從局部計畫中移除。此策略適用於原步驟與目前任務狀態不一致，或該步驟可能重複處理已完成操作的情境。

- **計畫調整：**系統會移除原失敗步驟，並使刪除後的計畫能銜接前後步驟繼續執行。此策略可表示為：

$$P_{\text{repair}} = \langle S_1, \dots, S_{k-1}, S_{k+1}, \dots, S_n \rangle$$

其中，原失敗步驟 S_k^{fail} 不再出現在修復後計畫中。

- **使用原則：**刪除策略主要處理計畫結構上的銜接，並不額外產生被刪除步驟原本可能提供的資料。因此，若後續執行過程顯示刪除造成必要資料不足，該失敗仍需透過後續局部修復或重新規劃處理。

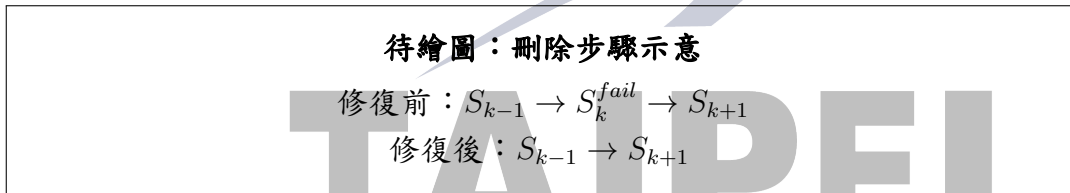


圖 3.4: 刪除步驟之局部計畫修改示意圖

案例待補：此處預計補入一個實際任務中的 *Delete* 成功案例，說明被刪除的冗餘或不適當步驟，以及刪除後如何銜接後續計畫。

整體而言，上述公式描述的是計畫序列層級的變化；實際執行時，系統仍需維持前後步驟與資料依賴關係的一致性。對應演算法 1，當 P_{repair} 產生後，系統會以其更新目前計畫 P ，並透過 $\text{SetNextStepToRepairPoint}(P)$ 將後續執行位置調整至修復後計畫中需要接續執行的位置。若後續執行仍出現資料不足或目標不符，系統會再依當前失敗狀態進入局部修復或重新規劃流程。

第四章 CBR-APIKGAgent 系統實作

本章說明 CBR-APIKGAgent 的系統實作方式。第三章已說明整體方法與任務執行流程，本章則依照系統實際運作時的資料準備與執行階段，說明原子化經驗庫如何建構，以及原子化經驗與局部修復機制如何落實於初始規劃、步驟執行、失敗處理與重新規劃流程中。

本研究使用 LangGraph 建立代理人控制流程，使用 Neo4j 與 API 知識圖譜支援 API 搜尋，並透過大型語言模型與嵌入模型完成任務規劃、經驗檢索、錯誤分析與局部修復。由於本研究重點在於原子化經驗重用與動態局部修復於多步驟 API 任務中的實作方式，以下不另以工具為主軸分節，而依系統資料準備與任務執行階段說明。

4.1 原子化經驗庫建構實作

原子化經驗庫於任務執行前建立，作為後續初始規劃、局部修復與重新規劃階段的經驗來源。本節僅說明經驗庫如何由訓練資料成功任務軌跡建構，至於各階段如何檢索與使用經驗，則於後續對應流程中說明。

4.1.1 訓練資料成功任務軌跡萃取

本研究使用訓練資料中的成功任務軌跡建立經驗庫。經驗萃取流程由經驗轉換模組負責，包含訓練任務資料讀取、成功軌跡整理，以及將任務證據轉換為原子化經驗等步驟。

訓練任務證據包含任務規格、公開資料、標準解答、評估條件與 API 呼叫紀錄等資訊。原子化經驗萃取器不要求每個訓練任務都產生經驗，而是要求模型只在成功軌跡中存在可跨任務重用的資料流風險、規劃原則或常見陷阱時，才產生一至數筆原子化經驗。此設計可降低將單一任務特定流程誤轉換為通用經驗的風險。

4.1.2 原子化經驗資料格式

第三章已從概念層級說明原子化經驗需保留來源脈絡、適用情境、核心原則、必要中間概念、套用方式與常見陷阱等資訊；本節進一步說明這些資訊在系統中的實際資料格式。原子化經驗以 JSONL 格式保存，每一行對應一筆經驗。本研究實作中使用的主要欄位包含：

- `experience_type`：經驗類型的粗略標籤，用於描述經驗所屬的策略類別。
- `source_task_id`：來源訓練任務識別碼。
- `source_task_goal`：來源任務目標，用於理解經驗來源情境。
- `when_to_use`：描述此經驗適合被考慮的任務或錯誤情境。
- `core_principle`：經驗欲保留的核心規劃原則或資料流不變條件。
- `required_intermediate_concepts`：規劃或修復過程中應保留的重要中間概念。
- `how_to_apply`：將經驗套用至新任務時可參考的抽象步驟。
- `pitfalls`：此經驗希望避免的常見錯誤。

上述欄位並不保存固定 API 呼叫序列，也不保存特定任務的參數值。系統在提示注入時會將這些欄位格式化為簡短策略提醒，提供給大型語言模型參考。

4.1.3 經驗向量化與雙索引建構

經驗檢索由語意檢索模組實作。系統啟動或索引更新時，會讀取離線經驗檔案，並為同一批經驗建立規劃導向與修復導向兩組向量。兩組向量使用相同經驗來源，但索引文字不同：

- **規劃導向索引**：由來源任務、適用情境、核心原則與套用方式等欄位組成，主要用於任務規劃相關情境。
- **修復導向索引**：由適用情境、核心原則、套用方式與常見陷阱等欄位組成，主要用於失敗處理與修復相關情境。

實作上，兩組索引會在任務執行前或索引更新時預先建立，使任務執行期間可直接依目前階段選擇對應索引進行查詢，而不需在每一次規劃或修復決策時重新組合經驗內容並產生向量表示。檢索時，系統會將查詢文字轉換為向量，並與對應索引中的經驗向量計算餘弦相似度。

4.1.4 經驗來源與索引使用

系統可區分訓練資料成功任務經驗與執行期間產生的局部修復紀錄。本研究實驗採用前者作為正式經驗庫來源；測試期間產生的修復紀錄不納入正式檢索引，以維持經驗來源與評估資料之間的切割。

在任務執行過程中，初始規劃、局部修復與重新規劃會依各自需求選擇規劃導向或修復導向索引。實際檢索數量與實驗控制設定於第五章實驗設定中說明。

待繪圖：原子化經驗庫建構流程

訓練資料成功任務軌跡 → 任務證據整理 → 原子化經驗萃取
→ JSONL 經驗庫 → 規劃導向索引／修復導向索引

圖 4.1: 原子化經驗庫建構流程圖（待繪）

圖 4.1 說明原子化經驗由訓練資料成功任務軌跡萃取、儲存至經驗庫，並建立兩種語意索引的流程。

4.2 初始規劃實作

任務開始後，系統先根據使用者任務搜尋候選 API，並結合規劃導向經驗產生初始任務計畫。本節說明初始規劃階段中 API 搜尋、經驗檢索與提示注入的實作方式。

4.2.1 API 搜尋與候選 API 整理

本研究使用 API 知識圖譜作為工具資訊來源。API 知識圖譜提供可用 API 及其描述、參數與回傳結構等資訊；系統會根據使用者任務搜尋與任務目標相關的候選 API，作為任務規劃器產生初始計畫時的工具依據。

候選 API 會以 API 名稱、功能描述、參數需求與回傳結構等形式提供給大型語言模型。此階段的 API 搜尋主要解決「有哪些工具可用」的問題；原子化經驗則提供「如何組織資料流程與避免常見錯誤」的策略參考。

4.2.2 規劃導向經驗檢索

初始規劃階段以使用者任務作為查詢，透過規劃導向索引取得相關原子化經驗，並將檢索結果格式化為精簡策略提醒。

檢索結果會提醒模型注意相似任務中的資料依賴、目標集合建構、候選驗證或輸出格式限制等問題。此處的經驗不作為固定解法，也不直接指定 API 呼叫序列，而是作為任務規劃時的策略參考。

4.2.3 初始計畫生成與提示注入

初始規劃提示包含使用者任務、候選 API 以及規劃導向經驗。提示會要求模型將經驗視為精簡策略參考，而非任務特定解法。

尤其當經驗包含集合、物件或中間概念等描述時，提示會要求模型將其理解為資料流語意，而不是直接使用 Python 特定資料型態或固定輸出格式。此設計可降低模型將經驗誤用為具體程式模板的風險。

待繪圖：初始規劃與經驗注入流程

使用者任務 → API 搜尋與候選 API 整理
使用者任務 → 規劃導向經驗檢索
候選 API + 規劃導向經驗 → 初始規劃提示
初始規劃提示 → 初始任務計畫

圖 4.2: 初始規劃與經驗注入流程圖（待繪）

圖 4.2 應呈現初始規劃階段中，API 搜尋與規劃導向經驗檢索如何分別提供工具資訊與策略參考，並共同形成任務規劃提示。

4.3 步驟執行與任務狀態管理

產生初始計畫後，系統依序執行計畫中的步驟。此階段的重點在於保存已完成步驟的執行結果、維持步驟間的資料依賴，並在失敗發生時提供後續錯誤處理所需的上下文。

4.3.1 步驟執行流程

系統主流程由 LangGraph 狀態圖控制。任務開始後，系統依序進入 API 搜尋、任務規劃、步驟控制與任務執行等節點。步驟執行節點會根據目前步驟類型執行 API 呼叫或資料處理動作，並將執行結果寫回任務狀態。

當步驟成功時，系統保存執行結果並前進至下一步；當步驟失敗時，條件路由會根據錯誤分類、失敗步驟類型、局部修復嘗試次數與修復鏈深度，決定是否進入局部修復節點或回退至重新規劃節點。

4.3.2 執行結果與相依資料保存

LangGraph 的狀態傳遞機制使各節點能共享任務目標、目前計畫、候選 API、執行結果、錯誤資訊與修復紀錄。成功步驟的輸出會保存於任務狀態中，作為後續步驟的相依資料；失敗步驟的錯誤訊息與相關上下文則會提供給失敗處理流程。

此設計使 CBR-APIKAgent 能在同一任務狀態下完成規劃、執行、局部修復與重新規劃，也使局部修復與重新規劃能取得已完成步驟的結果與失敗脈絡。

4.3.3 錯誤處理狀態資訊

除了原有的任務目標、候選 API、計畫、目前步驟與中間結果外，系統在失敗處理階段需要額外保存錯誤分析與局部修復相關資訊。表 4.1 以概念層級整理本研究方法中主要的狀態資訊類型；實際欄位名稱則依系統實作而定。

表 4.1: 錯誤處理與局部修復相關狀態資訊

資訊類型	用途	主要使用階段
暫定錯誤分類	保存錯誤分類流程產生的暫定類型，作為錯誤處理路由的輔助訊號。	錯誤分析、路由判斷
可行回饋	將錯誤訊息整理為可操作的修復建議，提供修復 API 搜尋與策略選擇參考。	修復 API 搜尋、策略選擇
結構化失敗資訊	保存失敗 API 與執行結果分析形成的結構化失敗資訊。	失敗證據整理、策略選擇
修復嘗試紀錄	記錄同一失敗脈絡已進行的局部修復嘗試，用於控制修復流程的延伸範圍。	路由判斷、修復限制檢查
修復步驟關聯	記錄修復步驟與原始失敗步驟之間的關係，使修復步驟失敗時仍能回到同一失敗脈絡判斷。	修復步驟追蹤、路由判斷
局部修復紀錄	保存局部修復過程中的主要決策資訊，例如選擇的修復方式與相關失敗證據。	修復紀錄、案例分析

4.4 失敗處理與局部修復實作

當步驟執行失敗時，系統會先整理錯誤資訊與相依資料，產生暫定錯誤分類與可行回饋；若錯誤處理路由判斷仍可嘗試局部修復，系統會搜尋修復 API、檢索修復導向經驗，並產生插入、替換或刪除等局部計畫修改。

4.4.1 錯誤分類與可行回饋產生

當任務執行器回報步驟失敗時，系統會先整理失敗步驟在目前計畫中的位置、步驟描述、原本欲完成的局部目標，以及 API 呼叫或資料處理所回傳的原始錯誤訊息。若該步驟與先前步驟存在資料依賴，系統也會保留可用的中間結果摘要與相依資料脈絡，使後續修復流程能判斷失敗是否可能與缺少前置資料、資料來源不一致或操作目標不完整有關。

在此基礎上，系統會根據失敗步驟、失敗 API 資訊、原始錯誤訊息與結構化失敗資訊，產生兩項輸出：暫定錯誤分類與可行回饋。前者描述目前失敗較接近的錯誤情境，後者將錯誤訊息整理為後續可用的修復建議。

系統僅將分類結果作為路由與修復輔助訊號；實際修復策略仍需結合候選 API、結構化失敗資訊與修復經驗進行判斷。

4.4.2 局部修復觸發與限制檢查

錯誤處理路由由條件路由流程控制。若步驟失敗且原始失敗步驟被暫定分類為不可行計畫，系統會直接回退至重新規劃。若失敗步驟可嘗試局部修復，且同一失敗點尚未達到修復嘗試上限，則進入局部修復流程。

若失敗的是先前新增或替換出的修復步驟，系統會透過修復步驟與原始失敗步驟的對應關係，將修復嘗試次數與修復鏈深度回掛至原始失敗點。此設計可避免修復步驟失敗後被視為全新的獨立錯誤，造成無限制擴張。

4.4.3 修復 API 搜尋與候選整理

系統會使用失敗訊息與可行回饋組成修復搜尋目標，並呼叫 API 搜尋流程取得候選修復 API。候選 API 會被整理為 API 名稱、描述與回傳結構等資訊，提供給後續修復策略選擇。

此階段的目的是不是直接修改計畫，而是取得可用工具。是否使用候選 API，以及候選 API 應插入、替換或刪除原步驟，仍由後續策略選擇流程決定。

4.4.4 修復導向經驗注入

取得候選 API 後，局部修復策略選擇流程會以失敗訊息查詢修復導向索引，取得與目前錯誤情境相關的經驗。

檢索到的修復經驗會與失敗步驟、錯誤訊息、可行回饋、候選 API 以及結構化失敗資訊一同提供給大型語言模型。此流程使修復經驗能在選擇插入、替換或刪除策略時提供資料流與常見陷阱方面的提醒。

4.4.5 插入、替換與刪除策略實作

局部修復策略選擇流程會根據失敗步驟、失敗訊息、可行回饋、候選 API、修復導向經驗與結構化失敗資訊，產生修復 API 選擇與計畫修改方式。此輸出會被轉換為實際計畫更新操作，而不是直接執行模型產生的文字說明。系統支援三種計畫修改方式：

- **插入：**在原失敗步驟前插入修復步驟，並保留原失敗步驟於修復步驟後繼續執行。此策略主要用於補足原步驟缺少的前置資料或狀態。計畫更新後，系統會維持後續步驟與位移後原失敗步驟之間的相依關係。
- **替換：**以新的修復步驟取代原失敗步驟，使原步驟不再執行。此策略主要用於原步驟本身使用不適當 API、操作目標錯誤，或原步驟的部分執行可能造成重複處理風險的情境。由於替換保留原步驟位置，後續原本依賴該步驟的流程可接續至新的修復步驟。
- **刪除：**移除原失敗步驟且不加入新的 API。此策略主要用於原步驟冗餘、重複或不應繼續執行的情境。刪除後系統會調整後續計畫的銜接，使任務能由刪除後的位置繼續執行；此處理主要維持計畫結構一致性，並不補足被刪步驟可能產生的資料。

策略選擇過程也會判斷修復步驟是否需要沿用原失敗步驟的前序相依資料。此設計使系統能依修復 API 的需求決定是否使用原計畫中已取得的中間結果，避免修復步驟在缺少必要資料的情況下被執行。

為避免模型過度修改原計畫，系統在套用策略前會進行保守檢查。當失敗證據不足以支持大幅改動時，系統傾向保留原有資料流；當原步驟本身可能造成重複操作或副作用時，則會避免讓該步驟再次執行。

待繪圖：局部修復與重新規劃回退流程

步驟失敗 → 錯誤分類與可行回饋 → 檢查修復限制
→ 修復 API 搜尋 → 修復導向經驗檢索
→ 選擇插入／替換／刪除 → 修改計畫並回到執行
若無法修復 → 回退至重新規劃

圖 4.3: 局部修復與重新規劃回退流程圖（待繪）

圖 4.3 呈現步驟失敗後的錯誤處理路由，特別是錯誤分類、修復限制檢查、修復 API 搜尋、修復導向經驗檢索、策略選擇與重新規劃回退之間的關係。

4.5 重新規劃實作

當局部修復不適用、修復嘗試次數或修復鏈深度達到限制，或局部修復無法形成可執行計畫修改時，系統會回退至重新規劃流程。重新規劃的目的，是在保留原始任務目標、既有計畫、已完成步驟與失敗資訊的基礎上，重新產生後續可執行計畫。

4.5.1 重新規劃觸發條件

若局部修復無法選出可用修復 API、無法形成計畫修改，或同一失敗脈絡已超出可控修復範圍，系統會將局部修復視為失敗，並回退至重新規劃流程。局部修復流程結束後，系統會依計畫是否已成功修改決定接續方向；若已有修復後計畫，則回到任務執行流程，否則進入重新規劃。

4.5.2 重新規劃提示內容

重新規劃階段會參考原始任務目標、既有計畫、目前失敗步驟、錯誤訊息與已完成步驟結果。相較於初始規劃，重新規劃提示需要同時保留任務原始需求與已觀察到的失敗情境，使重新產生的後續計畫能避開已發生的錯誤，並延續仍有效的資料流程。

4.5.3 規劃導向與修復導向經驗注入

當系統進入重新規劃時，會分別建立規劃導向查詢與修復導向查詢。規劃導向查詢包含任務目標、既有計畫、失敗步驟與錯誤訊息，用於取得與任務結構相關的經驗；修復導向查詢則著重於任務脈絡、失敗步驟與錯誤訊息，用於取得與失敗情境相關的經驗。

重新規劃階段會分別從規劃導向與修復導向索引取得經驗，並將兩類經驗注入重新規劃提示中。規劃導向經驗與修復導向經驗會分別呈現，使重新規劃器能同時參考任務結構與已觀察到的失敗情境。

4.5.4 重新規劃後的流程銜接

重新規劃節點產生修正後的後續計畫後，系統會重置與局部修復相關的暫存狀態，並將新計畫交回任務執行流程。此設計使系統能於局部修復與重新規劃之間切換：局部修復先嘗試以小範圍計畫修改恢復執行；當局部修復無法形成可執行修改或達到限制時，重新規劃節點會重新產生後續任務流程。



第五章 實驗與結果分析

本章旨在評估 CBR-APIKGAgent 於多步驟 API 任務中的任務完成能力與錯誤修復表現。前兩章已分別說明本研究之方法設計與系統實作；本章則透過 AppWorld 任務環境進行實驗，觀察原子化經驗重用與動態局部修復機制對整體任務完成率、模組貢獻與修復行為的影響。

由於本研究的核心假設為「由訓練資料成功任務軌跡萃取之原子化經驗，可作為相似任務結構或錯誤情境下的策略參考」，以及「局部修復可在部分失敗情境下協助代理人由執行錯誤中恢復」，因此本章將分別從整體效能、消融實驗與代表性案例等面向進行分析。實驗結果以實際執行紀錄與 AppWorld 評估結果為依據，並進一步搭配代表性任務觀察討論系統在不同情境下的表現與限制。

5.1 研究問題

本研究設計三項研究問題，以對應第一章所提出之研究目的：

1. RQ1：CBR-APIKGAgent 相較於既有方法在 AppWorld 任務中的整體表現為何？

此問題用於評估完整系統在 AppWorld test_normal 上的整體任務完成能力。

2. RQ2：原子化經驗重用機制與動態局部修復機制分別對任務完成率造成何種影響？

此問題用於透過消融實驗分析兩項核心機制對任務完成結果的個別影響。

3. RQ3：原子化經驗與動態局部修復在代表性成功與失敗案例中呈現哪些作用方式與限制？

此問題延續消融實驗結果，從不同設定間的成功與失敗差異中選取代表性任務，分析兩項機制如何使部分任務成功、退步或仍然失敗。

5.2 實驗設置

5.2.1 資料集與任務範圍

本研究於 AppWorld [3] 環境中進行實驗。AppWorld 提供多應用程式 API 任務環境，任務完成與否由最終環境狀態是否符合預期條件判定。因此，代理人不僅需要產生合理文字回覆，也必須透過 API 呼叫與資料處理正確改變或查詢環境狀態。

正式實驗以 AppWorld test_normal 作為主要評估資料集，並以本研究提出之 CBR-APIKGAgent 作為評估系統。AppWorld 採狀態式評估方式，亦即由最終環境狀態是否符合任務需求判定任務是否成功。各項實驗所使用的任務範圍將於對應實驗小節中說明。

5.2.2 評估指標

依據 AppWorld 官方評估方式，AppWorld 採用程式化且以最終狀態為基礎的 evaluation tests 判斷任務是否完成，並以任務目標完成率（Task Goal Completion, TGC）與情境目標完成率（Scenario Goal Completion, SGC）作為主要指標 [3]。

1. **任務目標完成率（TGC）**：TGC 表示通過所有 AppWorld evaluation tests 的任務比例。對於單一任務而言，只有當該任務的所有評估測試皆通過時，該任務才計為成功。其計算方式如下：

$$TGC = \frac{N_{\text{success}}}{N_{\text{total}}} \times 100\%$$

其中， N_{success} 代表通過所有 evaluation tests 的任務數量， N_{total} 代表評估任務總數。

2. **情境目標完成率（SGC）**：SGC 以 AppWorld 的 task scenario 為單位計算。只有同一 scenario 產生之所有任務皆通過 evaluation tests 時，該 scenario 才計為成功。此指標用於衡量代理人是否能在同一任務情境的不同需求與初始狀態變化下穩定完成目標。其計算方式如下：

$$SGC = \frac{N_{\text{success scenarios}}}{N_{\text{total scenarios}}} \times 100\%$$

5.2.3 實驗環境與主要控制設定

本研究實驗之硬體與軟體環境如表 5.1 所示。由於本研究主要透過雲端大型語言模型服務進行推論，本地端硬體主要負責執行 AppWorld 環境、代理人控制流程、API 模擬環境與實驗紀錄整理。

表 5.1: 硬體與軟體環境

項目	設定
作業系統	Ubuntu 22.04.5 LTS (WSL2)
處理器	12th Gen Intel(R) Core(TM) i7-12700
記憶體	16 GB
Python 版本	Python 3.12
AppWorld 版本	0.1.3

除上述執行環境外，實驗亦需固定語言模型、推論溫度、執行預算、經驗檢索設定與局部修復限制等主要控制項。表 5.2 列出本研究實驗中需要固定或比較的重點設定。

表 5.2: 實驗主要控制設定

項目	設定
大型語言模型	gemini-2.5-flash
溫度參數	一般判斷：0；規劃與重新規劃：0.2；執行推論：0.2；候選評分：0.1
任務執行時間限制	3600 秒 / 題
重新嘗試上限	3 次 / 題
局部修復上限	3 次 / 原始失敗步驟
單一步驟插入上限	2 個修復步驟 / 原始失敗步驟
經驗檢索數量	初始規劃：規劃導向 top-3；重新規劃：規劃導向 top-1、修復導向 top-1；局部修復：修復導向 top-2

5.3 實驗一：整體效能與失敗概況分析

本節對應 RQ1，目的在於呈現 CBR-APIKGAgent 在 AppWorld test_normal 上的整體任務完成率，並與既有方法進行比較。同時，本節亦整理完整系統中仍未通過任務的基本概況，作為後續消融實驗與案例研究的背景。本實驗使用 full test_normal 共 168 題任務，TGC 依通過評估之任務數除以 168 題計算。若任務執行後未產生完整評估報告或出現異常執行情形，該任務在統計中計為失敗。

為呈現本研究方法於 AppWorld 基準中的相對位置，表 5.3 列出既有方法於 test_normal 上的結果。其中，ReAct、PlanExec、FullCodeRefl 與 IPFunCall 之數據取自 AppWorld 原論文 [3]；LOOP 之數據取自 Chen 等人之研究 [17]；APIKGAgent 之數據取自 APIKGAgent 論文 [2]；CBR-APIKGAgent 則為本研究實驗結果。需注意的是，不同方法可能採用不同模型、提示設定或執行預算，因此表中結果主要作為基準表現參考。

表 5.3: 各方法於 AppWorld test_normal 之整體效能比較

模型	方法	TGC	SGC
GPT-4o	ReAct	48.8%	32.1%
	PlanExec	44.6%	23.2%
	FullCodeRefl	33.9%	26.8%
	IPFunCall	32.1%	16.1%
Llama3-70B	ReAct	20.8%	8.9%
	PlanExec	8.9%	1.8%
	FullCodeRefl	24.4%	17.9%
Qwen 2.5 32B	ReAct	39.2%	18.6%
	LOOP	72.6%	53.6%
Gemini 2.5 Flash	ReAct	50.6%	28.6%
	FullCodeRefl	36.9%	26.8%
	APIKGAgent	41.7%	17.9%
	CBR-APIKGAgent	47.62%	32.14%

為進一步呈現本研究完整系統於 test_normal 上的執行情形，表 5.4 將 168 題任務依評估結果拆分為成功、已產生評估報告但未通過，以及未產生完整評估報告三類。其中，未產生完整評估報告之任務可能來自執行超時、流程異常中斷或未順利進入評估階段；由於此類任務缺乏可確認之 AppWorld 評估結果，本研究在 TGC 統計中將其

計為未通過。

表 5.4: CBR-APIKGAgent 於 test_normal 之任務結果拆解

類別	數量	說明
成功任務	80	通過所有 AppWorld evaluation tests。
已評估但未通過任務	75	產生完整評估報告，但未通過所有 evaluation tests。
未產生完整評估報告任務	13	計入未通過任務。
總任務數	168	full test_normal。

由表 5.3 與表 5.4 可知，CBR-APIKGAgent 於 test_normal 上達成 47.62% 的 TGC 與 32.14% 的 SGC；相較之下，APIKGAgent 的 TGC 與 SGC 分別為 41.7% 與 17.9%。此結果顯示，在相同任務範圍與計分方式下，導入原子化經驗重用與動態局部修復後，整體任務完成率與情境完成率皆有所提升。不過，此整體差異尚無法單獨說明兩項機制各自的貢獻，因此下一節進一步透過消融實驗分析其影響。

從失敗概況來看，已評估但未通過任務代表系統完成執行並產生 AppWorld 評估報告，但最終環境狀態仍未滿足所有評估條件；未產生完整評估報告任務則表示執行流程未能順利進入或完成評估階段。前者較適合進一步分析任務理解、資料依賴、API 使用與修復策略等問題；後者則需搭配執行紀錄確認是否與 timeout、流程中斷或評估報告產生失敗有關。由於本研究主要關注原子化經驗與局部修復兩項機制的貢獻，後續將透過消融實驗與代表性案例進一步分析成功與失敗差異。

5.4 實驗二：消融實驗與案例分析

本節對應 RQ2 與 RQ3。首先，透過消融實驗分析原子化經驗重用機制與動態局部修復機制分別對系統表現造成的影響，以回答 RQ2。其次，本節延續消融實驗結果，從不同設定間的成功與失敗差異中選取代表案例，分析兩項機制在實際任務中的作用方式與限制，以回答 RQ3。

5.4.1 實驗組別

消融實驗共包含四種設定，用以分別觀察原子化經驗與局部修復對任務完成率的影響。四種設定皆固定在 AppWorld test_normal 中每個 scenario 的 _1 任務，共 56 題，並採相同計分方式。此設定使每個 scenario 在消融實驗中各提供一題任務，可避免特定 scenario 因包含多個變體而在模組貢獻分析中占較高比重。除上述機制差異外，其餘模型、時間限制、重試上限與評估方式皆保持一致。

5.4.2 消融實驗結果

正式消融實驗結果如表 5.5 所示。四種設定皆使用相同 56 題任務，完成率皆以 56 題為分母計算。

表 5.5: 消融實驗結果

設定	任務完成率
基礎系統：不使用原子化經驗與局部修復	39.29%
僅原子化經驗：使用原子化經驗，不使用局部修復	44.64%
僅局部修復：不使用原子化經驗，使用局部修復	48.21%
完整系統：同時使用原子化經驗與局部修復	46.43%

由表 5.5 可知，僅原子化經驗、僅局部修復與完整系統皆高於基礎系統，表示兩項設計皆可能對任務完成率帶來正向影響。然而，完成率差異不足以完整說明兩項機制是否作用於相同任務。因此，本節後續依三個層次進行討論：首先比較僅原子化經驗與基礎系統，以觀察原子化經驗的效益；其次比較僅局部修復與基礎系統，以觀察動態局部修復的效益；最後比較完整系統與其他設定，以觀察兩項機制同時啟用時的交互影響。

從任務層級交集來看，僅原子化經驗使 6 題基礎系統失敗任務轉為成功，僅局部修復使 10 題基礎系統失敗任務轉為成功，而完整系統則使 7 題基礎系統失敗任務轉為成功。此外，有 2 題任務僅在完整系統中成功，但也有 8 題任務在僅局部修復成功、完整系統未成功。此現象表示原子化經驗與局部修復皆能改善部分基礎系統失敗案例，但兩者並非單純線性疊加；經驗注入可能改變初始計畫、失敗位置或相依資料脈絡，使局部修復面對的錯誤條件不同。因此，完整系統未高於僅局部修復設定，應被解讀

為兩項機制存在交互影響，而非單一機制必然無效。

5.4.3 原子化經驗之效益分析

此部分比較基礎系統與僅原子化經驗設定之差異，觀察僅加入原子化經驗後，系統在任務規劃與重新規劃階段是否出現不同表現。原子化經驗的預期作用並非直接執行任務，而是提供可檢索的策略參考，使代理人在面對相似任務結構時能更早注意資料依賴、規劃順序或常見陷阱。除完成率差異外，本節亦關注檢索到的經驗是否能對應當前任務需求，以及經驗提示是否可能改變計畫方向。分析重點包含：

- 任務完成率是否相較基礎系統改變。
- 檢索經驗是否與任務目標、資料依賴或常見陷阱相符。
- 經驗提示是否有助於減少已知失敗模式在相似任務中出現。
- 經驗是否可能造成誤導，例如檢索到情境相似但實際不適用的策略。

僅原子化經驗設定在 56 題 subset 中任務完成率為 44.64%；相較於基礎系統的 39.29%，提升 5.36 個百分點。此結果顯示，在關閉局部修復的條件下，原子化經驗作為規劃與重新規劃階段的策略參考，對部分任務具有正向影響。不過，此結果仍需搭配檢索內容與任務結果觀察判斷其實際作用，因為檢索經驗也可能因情境相似但細節不同而造成有限或不適用的參考。

5.4.4 動態局部修復之效益分析

此部分比較基礎系統與僅局部修復設定之差異，觀察僅加入動態局部修復後，系統是否能在部分執行失敗後恢復。局部修復的預期作用是在不重新產生完整計畫的情況下，根據失敗步驟、錯誤訊息與候選 API 對局部計畫進行調整。分析重點包含：

- 局部修復觸發次數與實際修改計畫次數。
- 插入、替換與刪除三種策略的使用情形。
- 局部修復後任務是否繼續執行，以及是否最終成功。

僅局部修復設定在相同 56 題 subset 中任務完成率為 48.21%；相較於基礎系統的 39.29%，提升 8.93 個百分點。此結果顯示，在不使用訓練資料原子化經驗的條件下，動態局部修復仍能使部分原本失敗的任務恢復執行並最終完成。與僅原子化經驗設定相比，僅局部修復設定高出 3.57 個百分點，表示在此 56 題 subset 中，局部修復對完成率的直接影響較為明顯。

5.4.5 兩項機制之綜合效益分析

此部分比較完整系統與其他消融設定之差異，觀察原子化經驗與局部修復同時啟用時是否產生額外影響。由於原子化經驗會影響規劃與重新規劃，而局部修復則作用於執行失敗後的計畫調整，兩者可能互補，也可能因改變失敗位置與資料脈絡而產生非線性影響。分析重點包含：

- 完整系統相較基礎系統的任務完成率差異。
- 完整系統相較單一機制設定的變化。
- 經驗是否在局部修復策略選擇時提供有用參考。

完整系統在 56 題 subset 中任務完成率為 46.43%；相較於基礎系統提升 7.14 個百分點。與僅原子化經驗設定相比，完整系統高出 1.79 個百分點；但與僅局部修復設定相比，完整系統低 1.79 個百分點。此結果顯示，原子化經驗與局部修復在部分任務中可能具有互補效果，但兩者結合並不保證產生線性加成。可能原因是經驗提示會改變初始計畫、失敗位置或相依資料脈絡，使局部修復面對的錯誤情境與「僅局部修復」設定不同。因此，完整系統的效果需進一步搭配任務交集與案例分析解釋，而不宜僅以單一完成率判斷兩項機制必然互補。

5.4.6 消融實驗案例分析

本節對應 RQ3，並非新增獨立實驗，而是延續實驗二之消融結果進行案例分析。其目的在於從不同設定間的成功與失敗差異中選取代表性任務，說明原子化經驗與動態局部修復如何影響實際任務的成功、失敗與退步情形。與前一節的完成率比較不同，

本節著重於任務層級的行為觀察：哪些任務因加入經驗或局部修復而被救回，哪些任務在完整系統中仍然失敗，以及兩項機制同時存在時為何不一定呈現線性加成。

案例選取將以實驗二的任務交集為依據，優先分析以下四類任務：

- 基礎系統失敗、僅原子化經驗成功的任務，用於觀察經驗是否改善規劃或重新規劃。
- 基礎系統失敗、僅局部修復成功的任務，用於觀察局部修復是否協助代理人由執行錯誤中恢復。
- 僅完整系統成功的任務，用於觀察兩項機制可能互補的情形。
- 僅局部修復成功但完整系統失敗的任務，用於分析經驗提示是否改變初始計畫、失敗位置或相依資料脈絡。

【待分析】 本節正式案例尚待由最終實驗 log、metadata、LLM history 與 evaluator report 整理後補入。以下先保留案例分析架構，不對個別任務的錯誤原因做結論性描述。

5.4.6.1 原子化經驗改善案例

本節將選取基礎系統失敗、僅原子化經驗成功的代表性任務，分析檢索到的原子化經驗如何影響初始規劃或重新規劃。案例分析建議包含任務目標、檢索到的經驗重點、計畫差異，以及最終評估結果。

【待分析】 待補入代表性案例。建議優先挑選可清楚呈現經驗如何提醒資料依賴、規劃順序或常見陷阱的任務。

5.4.6.2 局部修復改善案例

本節將選取基礎系統失敗、僅局部修復成功的代表性任務，說明局部修復如何根據失敗訊息、候選 API 與相依資料調整局部計畫。案例分析建議包含原始失敗步驟、修復策略、修復前後計畫差異，以及任務最終結果。

【待分析】 待補入代表性案例。建議至少選取一個可清楚呈現插入、替換或刪除策略如何協助任務恢復的案例。

5.4.6.3 兩項機制互補案例

本節將選取僅完整系統成功的代表性任務，分析原子化經驗與局部修復如何在同一任務中形成互補效果。此類案例可用於說明經驗提供的策略參考如何影響後續修復情境，或局部修復如何補足經驗提示後仍未完全解決的執行問題。

【待分析】待補入代表性案例。建議挑選實驗二中僅完整系統成功的任務，以避免案例解釋與消融結果脫節。

5.4.6.4 退步與仍失敗案例

本節將選取完整系統未成功的代表性任務，分析兩項機制的的能力限制。可能情境包含經驗檢索與當前任務僅表面相似、經驗提示改變初始計畫導致失敗位置改變、局部修復無法補足後續資料依賴，或環境狀態已被前序步驟改變而難以恢復。

【待分析】待補入代表性案例。建議至少選取一個「僅局部修復成功但完整系統失敗」的任務，用於說明兩項機制同時使用時可能產生的非線性影響。

5.5 討論

5.5.1 原子化經驗的影響與適用情境

根據 RQ2 消融結果，僅啟用原子化經驗時，任務完成率由基礎系統的 39.29% 提升至 44.64%。此結果支持原子化經驗作為規劃與重新規劃階段策略參考的可能性，特別是在任務結構、資料依賴或常見陷阱與訓練資料中成功軌跡相似時，經驗可能協助代理人形成較合適的初始計畫或修正方向。

然而，完整系統並未高於僅局部修復組，顯示經驗並非在所有情境下都會帶來額外增益。經驗檢索結果可能改變計畫路徑，使後續錯誤與局部修復條件發生變化；也可能因經驗與當前任務僅表面相似，而未能提供有效參考。因此，原子化經驗的影響應被理解為一種可檢索的策略輔助，而非保證提升任務完成率的機制。

5.5.2 動態局部修復的影響與適用情境

根據 RQ2 消融結果，僅啟用局部修復時，任務完成率由基礎系統的 39.29% 提升至 48.21%，為四種消融設定中最高。此結果顯示，在部分失敗情境下，透過插入、替換或刪除步驟調整局部計畫，確實可能協助代理人由執行錯誤中恢復。

不過，局部修復的效果仍受限於失敗當下可取得的錯誤訊息、候選 API 搜尋結果與既有資料依賴。若錯誤來自整體任務理解偏差、前序狀態已不可逆地改變，或修復後仍無法滿足後續步驟所需資料，局部修復可能無法改善最終結果。因此，待分析之 RQ3 案例分析將進一步檢視原子化經驗與局部修復在實際任務中的成功、退步與失敗情形。

5.5.3 系統限制與能力邊界

本研究未將局部修復過程中產生的修復紀錄納入正式經驗庫，主要原因在於修復紀錄的可靠性不易由單一步驟結果判斷。部分修復雖可能使系統繼續執行，但未必能導向正確的任務完成結果；若直接將此類紀錄作為可重用經驗，可能使後續任務檢索到不可靠的策略，進而干擾規劃或修復決策。因此，如何判斷修復經驗是否具有可重用價值，是本研究尚未處理的限制之一。

此外，由於本研究在 AppWorld 任務中不假設可回復至錯誤發生前的環境狀態，局部修復與重新規劃皆是在目前已觀察到的狀態下繼續執行。此設定較貼近不可任意回滾的 API 任務情境，但也使部分已造成外部狀態改變的錯誤難以完全透過局部修復處理。

正式限制分析將於實驗結果與案例整理完成後補充。

第六章 結論與未來研究

6.1 結論

6.1.1 研究成果總結

6.1.2 研究貢獻

6.1.3 研究限制

6.2 未來研究

6.2.1 修復經驗累積機制

未來可進一步設計修復經驗驗證與累積機制，使系統在確認任務最終成功後，回溯分析局部修復步驟是否對成功結果具有正向貢獻，再決定是否將該修復紀錄轉換為可重用經驗。透過成功驗證、錯誤過濾與品質評分等方式，可降低不可靠修復經驗進入經驗庫的風險，並進一步探討經驗庫隨任務執行逐步擴充的可能性。

6.2.2 多輪局部修復策略

6.2.3 約束感知修復機制

6.2.4 跨基準資料集驗證

參考文獻

- [1] L. Wang, C. Ma, X. Feng, *et al.*, “A survey on large language model based autonomous agents,” *Front. Comput. Sci.*, vol. 18, no. 6, 186345, Mar. 2024. DOI: 10.1007/s11704-024-40231-1.
- [2] 吳柏諭，一個由 LLM 驅動之 AI 代理人透過操作 API 自動完成任務之研究，碩士論文，國立臺北科技大學資訊工程系，2025。 <https://hdl.handle.net/11296/837n46>.
- [3] H. Trivedi, T. Khot, M. Hartmann, *et al.*, “AppWorld: A controllable world of apps and people for benchmarking interactive coding agents,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 16 022–16 076. DOI: 10.18653/v1/2024.acl-long.850.
- [4] J. S. Park, J. O’Brien, *et al.*, “Generative agents: Interactive simulacra of human behavior,” *Proceedings of the ACM Symposium on User Interface Software and Technology*, 2023.
- [5] S. Yao, J. Zhao, D. Yu, *et al.*, “React: Synergizing reasoning and acting in language models,” *International Conference on Learning Representations (ICLR)*, 2023.
- [6] L. Wang, W. Xu, *et al.*, “Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models,” *arXiv preprint arXiv:2305.04091*, 2023.
- [7] N. Shinn, F. Cassano, *et al.*, “Reflexion: Language agents with verbal reinforcement learning,” *Advances in Neural Information Processing Systems*, 2023.
- [8] M. Li *et al.*, “Toolbench: Towards open tool learning for large language models,” *Advances in Neural Information Processing Systems*, 2023.
- [9] Y. Qin *et al.*, “Toolllm: Facilitating large language models to master 16000+ real-world apis,” *Advances in Neural Information Processing Systems*, 2023.
- [10] A. Aamodt and E. Plaza, “Case-based reasoning: Foundational issues, methodological variations, and system approaches,” *AI Communications*, vol. 7, no. 1, pp. 39–59, 1994. DOI: 10.3233/AIC-1994-7104.
- [11] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, “ExpeL: LLM agents are experiential learners,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, 2024, pp. 19 632–19 642. DOI: 10.1609/aaai.v38i17.29936.
- [12] V. Karpukhin, B. Oguz, S. Min, *et al.*, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2020, pp. 6769–6781. DOI: 10.18653/v1/2020.emnlp-main.550.

- [13] P. Lewis, E. Perez, A. Piktus, *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [14] A. Madaan *et al.*, “Self-refine: Iterative refinement with self-feedback,” *Advances in Neural Information Processing Systems*, 2023.
- [15] K. Zhu, Z. Liu, B. Li, *et al.*, “Where LLM agents fail and how they can learn from failures,” *arXiv preprint arXiv:2509.25370*, 2025.
- [16] Z. Chen *et al.*, “Divide-verify-refine: Self-correction via verification and refinement for complex tasks,” *arXiv preprint*, 2024.
- [17] K. Chen *et al.*, “Reinforcement learning for long-horizon interactive LLM agents,” *arXiv preprint arXiv:2502.01600*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.01600>.



附錄

6.3 Prompts

在此整理撰寫論文、設計實驗與系統實作時使用的 prompts。

6.4 程式碼

在此整理重要程式碼片段與關鍵實作說明。

